

MODULE 7 · FOUNDATIONAL MODEL FINE-TUNING

Foundational Model Fine-Tuning

PEFT · Instruction Tuning · RLHF · Data Preparation & Evaluation

Concepts 10–27

Topic Range

B3–B4 Bloom's

Apply / Analyse

L2–L3

Intermediate–Advanced

Learning Objectives

Adapter Modules

Design & apply Housby/Pfeiffer adapters; compare with LoRA; choose based on task & hardware requirements

PEFT Trade-offs

Evaluate LoRA, QLoRA, Prefix Tuning, Adapters across 6 dimensions; apply hardware-driven decision framework

Instruction Tuning

Build quality (instruction, input, output) datasets; apply LIMA quality principles; execute multi-task IT

RLHF Pipeline

Implement SFT→RM→PPO; derive Bradley-Terry loss; identify and mitigate reward hacking

Data Preparation

Apply 5-stage data cleaning pipeline; select appropriate evaluation layer for each development stage

End-to-End Production

Trace: Base model → QLoRA SFT → DPO alignment → HuggingFace Hub → LLMops deployment path

Module 7 — Five Sub-Modules at a Glance



Each sub-module builds on the previous — fine-tuning is a layered stack from PEFT efficiency through alignment to deployment-ready evaluation.

Adapter Modules & PEFT Efficiency Trade-offs

Bottleneck FFN layers — base frozen, adapters trained

A structured framework: LoRA · QLoRA · Prefix Tuning · Adapters

What Is Parameter-Efficient Fine-Tuning (PEFT)?

Definition

PEFT is a family of methods that fine-tune only a small subset of model parameters (0.01–4% of total) while keeping the base model weights frozen. This achieves task-specific adaptation at a fraction of the compute and memory cost of full fine-tuning.

Why PEFT Matters

- Full fine-tuning a 7B model requires ~80 GB VRAM and days of compute
- PEFT reduces trainable parameters by 10–1000×, enabling single-GPU training
- Separate task adapters share one frozen base — multi-task efficiency
- Catastrophic forgetting is mitigated — base weights remain unchanged

The PEFT Spectrum

- LoRA/QLoRA — low-rank decomposition of weight matrices ($A \cdot B$)
- Adapter Modules — bottleneck FFN layers inserted into Transformer blocks
- Prefix Tuning — learnable tokens prepended to attention context
- Prompt Tuning — soft prompt vectors optimised at the input layer

Adapter Modules — Formal Definition (Houlsby et al., ICML 2019)

Definition (Houlsby et al., ICML 2019)

Adapter modules are small bottleneck feed-forward networks inserted into each Transformer block, with the base model weights completely frozen. Only the adapter parameters (~0.5–4% of total) are trained.

- Architecture: down-projection $W_{\text{down}} \in \mathbb{R}^{d \times k} \rightarrow$ non-linearity (GELU/ReLU) $\rightarrow$ up-projection $W_{\text{up}} \in \mathbb{R}^{k \times d}$, with a skip connection back to input h
- W_{up} is initialised to all-zeros so $\text{adapter}(h) \approx 0$ at training start — preserving the base model's behaviour exactly
- Bottleneck dimension $k \ll d$ is the key parameter budget lever ($d=768, k=64 \rightarrow \sim 0.5\%$ of BERT-base parameters)
- Two insertion positions per Transformer block: after multi-head attention AND after the feed-forward sublayer
- Base model weights are completely frozen throughout — only adapter parameters receive gradient updates during fine-tuning

Adapter Architecture — Transformer Block with Adapters

Transformer Block

Multi-Head Attention (FROZEN)

Adapter 1 ← (TRAINED)

Layer Norm (FROZEN)

Feed-Forward Network (FROZEN)

Adapter 2 ← (TRAINED)

Adapter Module Internals

Input $h \in \mathbb{R}^d$ (from frozen layer)

$W_{\text{down}} \in \mathbb{R}^{d \times k}$ down-project to k

GELU / ReLU activation

$W_{\text{up}} \in \mathbb{R}^{k \times d}$ up-project to d

+ Residual skip from input h

Output $h' = h + \text{adapter}(h)$

Near-Zero Initialisation — Why It Preserves Base Model Behaviour

Key Insight

W_{up} is initialised to all zeros so $\text{adapter}(h) \approx 0$ at training start. Therefore $h' = h + 0 = h$ — the adapter acts as an identity function initially, leaving the frozen base model's representations completely intact.

Without Near-Zero Init

Random W_{up} introduces noise into every layer from step 1. The SFT checkpoint's learned representations are destroyed before any task-specific gradient arrives.

With Near-Zero Init ($W_{up} = 0$)

Training starts from the frozen model's exact output. The adapter learns incremental corrections rather than fighting random initialisation noise throughout early training.

The Residual Connection Role

$h' = h + \text{adapter}(h)$ ensures the original signal always passes through unchanged. The adapter only contributes an additive correction term on top of the base representations.

Training Dynamics Impact

Near-zero init dramatically improves stability in early epochs. The task loss gradient can guide learning from a clean, predictable starting point at every layer.

Bottleneck Dimension — Parameter Count Calculation

$$\text{Params per Adapter} = d \times k \text{ (W_down)} + k \text{ (bias)} + k \times d \text{ (W_up)} + d \text{ (bias)} \approx 2dk \text{ when } k \ll d$$

Configuration	Calculation
d=768 (BERT-base), k=64	W_down: $768 \times 64 = 49,152$ W_up: $64 \times 768 = 49,152$ biases: 832 Total: ~99,136
2 adapters per block, 12 blocks	$12 \times 2 \times 99,136 = 2,379,264$ adapter params
BERT-base total	~110,000,000 params
Adapter fraction (k=64, Houlsby)	$2,379,264 / 110,000,000 = 2.16\%$
Pfeiffer config (k=64, 1 adapter/block)	$12 \times 1 \times 99,136 = 1,189,632 \rightarrow 1.08\%$
k=16, Pfeiffer	$12 \times (768 \times 16 + 16 \times 768) = 294,912 \rightarrow 0.27\%$
General formula	$2dk \times L \times \text{insertions_per_block}$

Practical guideline: start with k=64 for quality, reduce to k=16 for minimal overhead on constrained hardware.

Two Insertion Positions per Transformer Block

Position 1 — After Multi-Head Attention

- Inserted between MHA sublayer output and the residual add operation
- Adapts how the model uses attention-derived contextual representations
- Particularly effective for sequence labelling and relation classification tasks
- Housby original: placed after MHA before layer norm application
- Gradient flows: task loss → adapter 1 → frozen MHA (MHA receives no gradient)

Position 2 — After Feed-Forward Network

- Inserted between FFN sublayer output and the residual add operation
- Adapts the token-level representation transformation learned by the FFN
- Pfeiffer config: only this position — reduces params $\sim 4\times$ vs dual-adapter
- Empirically: single adapter after FFN often matches dual-adapter on GLUE
- Lower inference overhead than Housby — preferred in production deployments

Architecture insight: Pfeiffer single-position drops from $\sim 3.6\%$ to $\sim 0.8\%$ params with minimal GLUE accuracy loss (~ 0.3 points).

Adapter Mathematics — Annotated Forward Pass Equations

$$\mathbf{h} \in \mathbb{R}^d$$

Input hidden state from the frozen sublayer (d = model hidden size: 768 BERT-base, 4096 Llama-7B, 8192 Llama-70B)

$$\mathbf{z} = \mathbf{W}_{\text{down}} \cdot \mathbf{h} + \mathbf{b}_{\text{down}}$$

Down-projection $\mathbb{R}^d \rightarrow \mathbb{R}^k$: compresses to bottleneck k . $\mathbf{W}_{\text{down}}$ shape is (d, k) . This is the information bottleneck.

$$\mathbf{a} = \mathbf{f}(\mathbf{z}) = \text{GELU}(\mathbf{z})$$

Non-linear activation. $\text{GELU}(x) = x \cdot \Phi(x)$ where Φ is Gaussian CDF. ReLU is also used. Applied element-wise.

$$\mathbf{u} = \mathbf{W}_{\text{up}} \cdot \mathbf{a} + \mathbf{b}_{\text{up}}$$

Up-projection $\mathbb{R}^k \rightarrow \mathbb{R}^d$: restores to model hidden size. $\mathbf{W}_{\text{up}}$ shape is (k, d) . Initialised to zeros at training start.

$$\mathbf{h}' = \mathbf{h} + \mathbf{u}$$

Residual connection: original signal $\mathbf{h}$ plus adapter correction $\mathbf{u}$. At initialisation: $\mathbf{u} \approx 0$ so $\mathbf{h}' \approx \mathbf{h}$ (identity function).

Traced Example — Forward Pass Through Adapter (d=768, k=64)

Input	$h \in \mathbb{R}^{768}$	Token representation from frozen MHA sublayer, e.g. $h=[0.23, -0.11, \dots, 0.87]$ — 768 floating point values
Down-project	$z = W_{\text{down}} \cdot h \in \mathbb{R}^{64}$	768→64 compression via matrix multiply. W_{down} shape: (768, 64). Information bottleneck.
Activate	$a = \text{GELU}(z) \in \mathbb{R}^{64}$	Smooth non-linearity. $\text{GELU}(x)=x \cdot \Phi(x)$. Applied element-wise to 64-dim bottleneck.
Up-project	$u = W_{\text{up}} \cdot a \in \mathbb{R}^{768}$	64→768 expansion. W_{up} shape: (64,768). At training start $W_{\text{up}}=0$ so $u=0$.
Residual	$h' = h + u \in \mathbb{R}^{768}$	Add skip connection. At init: $u \approx 0 \rightarrow h' \approx h$. After training: u encodes task-specific adaptation.

The Adapter Variant Family — Five Key Designs

Houlsby (Original, ICML 2019)

Two adapters per block (post-MHA + post-FFN).
~3.6% params. Highest quality baseline for BERT multi-task. Reference implementation.

Pfeiffer Adapter (EACL 2021)

Single adapter after FFN only. ~0.8% params.
Simpler with minimal quality loss. Preferred when inference overhead matters.

AdapterFusion

Learnable composition of N task-specific adapters. Combines them without retraining any adapter — ideal for multi-task serving.

Compacter (NeurIPS 2021)

Hypercomplex (Kronecker product) multiplication replaces dense matrix multiply.
~0.05% params — extreme compression for constrained budgets.

Llama-Adapter

Zero-init attention adapters at upper Transformer layers only. <1% params. Designed for fast Llama-style instruction tuning with minimal overhead.

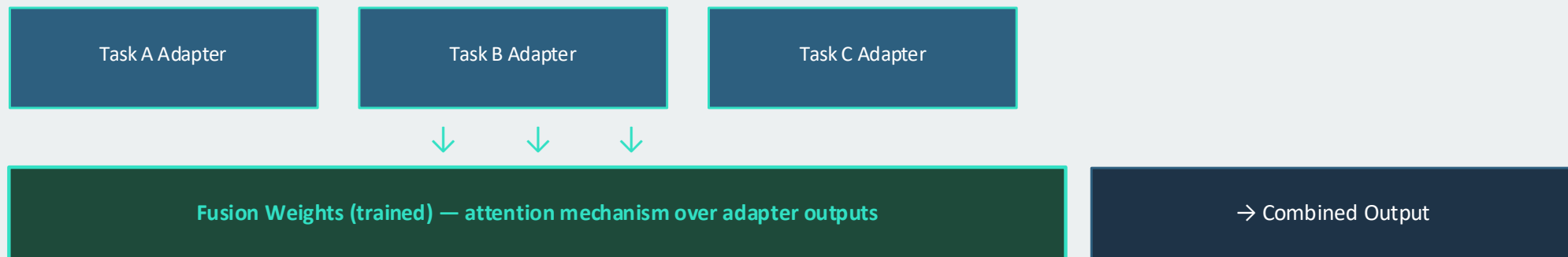
Adapter Variants — Full Comparison Table

Variant	Key Innovation	Params	Best Use Case	Library
Houlsby (original)	Adapters after attention + FFN (dual position)	~3.6%	Multi-task BERT; quality baseline	AdapterHub
Pfeiffer Adapter	Single adapter after FFN only	~0.8%	Lower overhead; minimal quality loss	AdapterHub
AdapterFusion	Learnable composition of N pre-trained adapters	$N \times \text{adapter}$	Multi-task without retraining existing adapters	AdapterHub
Compacter	Hypercomplex (Kronecker) multiplication	~0.05%	Extreme parameter budget constraint	Custom impl.
Llama-Adapter	Zero-init attention at upper layers only	<1%	Instruction tuning with minimal params	HF PEFT

AdapterFusion — Non-Destructive Multi-Task Composition

AdapterFusion (Pfeiffer et al., EACL 2021)

AdapterFusion learns a small set of composition weights that combine N pre-trained task adapters at inference time. No task adapter is retrained — only the fusion layer weights are learned. This enables serving many tasks from a single frozen base model.



- Each task adapter trained independently then frozen — no forgetting between tasks
- Fusion adds $O(k^2)$ parameters per block, much smaller than any individual adapter
- At inference: all task adapters contribute, weighted by the learned fusion attention
- New task: train new adapter independently, add to fusion pool — no retraining of existing adapters

Compacter — Hypercomplex Compression to 0.05% Parameters

Compacter (Mahabadi et al., NeurIPS 2021)

Compacter replaces standard matrix multiplications in adapter layers with parameterised hypercomplex (PHM) multiplications using Kronecker products, achieving $\sim 72\times$ compression versus Houlsby adapters with competitive GLUE performance.

Standard Adapter Matrix (W_{down})

- Shape: $\mathbb{R}^{768 \times 64} = 49,152$ parameters per matrix
- Standard dense matrix multiplication
- Full unconstrained rank bottleneck projection
- Memory: $4 \text{ bytes} \times 49,152 = \sim 196 \text{ KB}$ per matrix

Compacter PHM Matrix

- Reconstructed via Kronecker product: $A \otimes B$
- $A \in \mathbb{R}^{n \times n}$, $B \in \mathbb{R}^{(d/n) \times (k/n)}$ — two small shared matrices
- Hypercomplex number $n=4$: $n^2 + (d/n) \times (k/n) = 16 + 192 \times 16 = 3,088$ params
- $\sim 0.05\%$ of BERT-base vs 0.5% for standard Houlsby

Use case: embedded devices, federated learning, very constrained VRAM inference where every parameter must be justified.

Adapter Modules vs LoRA — Practical Comparison

Adapter Modules

- Adds new layers to architecture → forward pass overhead +5 to 15%
- Fully separable: swap task adapter per forward pass (AdapterFusion)
- Standard insertion: 1–2 positions per Transformer layer
- Best for: multi-task settings, BERT-era models, server-side inference
- Native support: AdapterHub library (adapterhub.ml)
- No weight merging needed — adapters stay separate at inference time

LoRA (Low-Rank Adaptation)

- No new layers — modifies weight matrices via low-rank $A \cdot B$ decomposition
- Zero inference overhead after merging: $W' = W + A \cdot B$ merged into W
- Targets: Q, K, V, O, FFN up/down projection matrices
- Best for: generative LLMs (Llama, Mistral), on-device inference
- Native support: HuggingFace PEFT library (github.com/huggingface/peft)
- Multi-task: keep separate (LoRAX) or merge into base for production

Decision: When Adapters Win vs When LoRA Wins

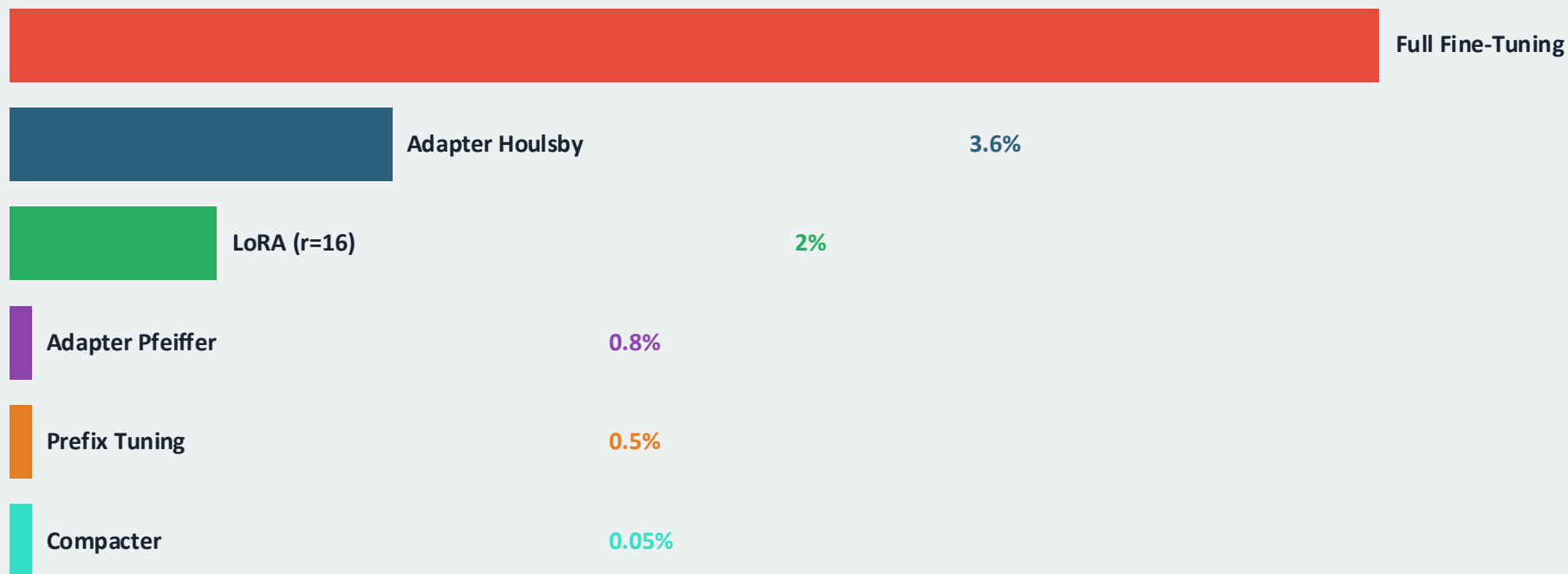
Criterion	Choose Adapters	Choose LoRA
Inference latency	Tolerates +5–15% overhead	Zero overhead required (merge)
Model family	BERT/RoBERTa encoder-only	Llama/Mistral/GPT decoder-only
Multi-task serving	AdapterFusion — compose N tasks	Multi-LoRA with LoRAX server
Task type	Classification, NLI, NER, span extraction	Generation, chat, code, summarisation
Hardware	Server GPU (A100, H100)	Consumer GPU (RTX 3090/4090)
Library maturity	AdapterHub — rich BERT ecosystem	HF PEFT — broader generative community
Parameter budget	0.5–4% (more flexible)	0.1–4% (comparable range)

PEFT Methods — 4-Dimension Efficiency Comparison

Dimension	LoRA	QLoRA	Prefix Tuning	Adapters
Trainable params	0.1–4%	0.1–4% (4-bit base)	0.01–1%	0.5–4%
Inference overhead	None (merged)	None (merged)	Context tokens +latency	5–15% latency
Task performance	★★★★★	★★★★½	★★★★☆☆	★★★★☆
Implementation ease	Easy (PEFT lib)	Moderate (BnB)	Moderate	Moderate (AdapterHub)
Multi-task serving	Multi-LoRA / LoRAX	Multi-LoRA / LoRAX	Swap prefix vector	AdapterFusion
Best hardware	1× A10G / RTX 4090	1× RTX 3090/4090 (≤ 24 GB)	Any GPU	Server GPU A100+

Trainable Parameter Budget — Visual Comparison

% Trainable Parameters (log scale — bar width $\propto \log_{10}(\% + 0.01)$)



Key insight: PEFT methods require 25–2,000× fewer trainable parameters than full fine-tuning, enabling single-GPU training of models that would otherwise require multi-node A100 clusters.

Inference Overhead Deep-Dive

Adapters — Architectural Latency Cost

- Extra forward pass through $W_{\text{down}} + W_{\text{up}}$ at every Transformer block
- For BERT-base (12 blocks $\times$ 2 adapters): 24 extra FFN passes per inference
- Typical latency increase: +5–15% per forward pass depending on k
- Overhead is fixed regardless of sequence length — constant cost per layer
- Cannot be eliminated by weight merging (architecturally distinct modules)
- Pfeiffer single-adapter halves the overhead vs dual-adapter Houlsby

LoRA — Zero Overhead Post-Merge

- During training: $A \cdot B$ computed separately and added to sublayer output
- After training: merge $W' = W + A \cdot B$ — single standard weight matrix
- No extra parameters at inference — behaviour identical to full fine-tuned model
- Merge: `model.merge_and_unload()` in HuggingFace PEFT
- Multi-task: keep adapters separate, switch per request (LoRAX in microseconds)
- Production default: merge for single-task, keep separate for multi-task API

Production rule: latency-critical SLA $\rightarrow$ LoRA (merged). Serving many tasks simultaneously $\rightarrow$ adapters + AdapterFusion may be more efficient overall.

Task Performance Trade-offs — PEFT Methods vs Full Fine-Tuning

Method	GLUE Avg (BERT)	MT-Bench Score	HumanEval pass@1	Relative to Full FT
Full Fine-Tuning	89.4	7.9	52.3%	Baseline (100%)
LoRA (r=16)	88.9	7.7	51.8%	~99% of Full FT
QLoRA (4-bit, r=16)	88.1	7.5	50.2%	~97% of Full FT
Adapter Houlby (k=64)	88.5	N/A	N/A	~98% of Full FT
Adapter Pfeiffer (k=64)	87.8	N/A	N/A	~96% of Full FT
Prefix Tuning (100 tokens)	84.2	6.8	43.1%	~88% of Full FT
Prompt Tuning	82.1	6.2	38.4%	~83% of Full FT

Numbers are representative ranges from published benchmarks. Actual results vary by task, model family, learning rate, and hyperparameter tuning.

Hardware-Driven Decision Framework — Choose Your PEFT Method

IF: ≤ 24 GB VRAM (RTX 3090/4090)

→ **QLoRA**

Load base in 4-bit (BitsAndBytes). Apply LoRA adapters on top. Default for most practitioners working on consumer hardware.

IF: 40–80 GB VRAM (A100 40/80GB)

→ **LoRA (bf16)**

Full-precision base + LoRA adapters. Slightly higher quality than QLoRA. Recommended for production-quality SFT pipelines.

IF: Constrained generation task

→ **Prefix Tuning**

Table-to-text, structured summarisation, constrained format. Best when model input structure is the primary control lever.

IF: Multi-task BERT-era encoder

→ **Adapters + Fusion**

Serving many NLI/classification tasks from one BERT/Roberta base. AdapterFusion enables dynamic task composition.

IF: Zero inference overhead required

→ **LoRA (merged)**

Merge adapters into base weights after training via `model.merge_and_unload()`. No overhead at inference. Best for production.

IF: Maximum quality, sufficient compute

→ **Full Fine-Tuning**

Large task dataset + A100 cluster. Only when PEFT quality gap is unacceptable. Rare in modern practice.

Multi-Adapter Serving — The Problem Without PEFT Fusion

Scenario: Serving 10 NLP Tasks With a 70B LLM (Naive Approach)

Without multi-adapter serving: 10 separate full fine-tuned model copies. Each 70B model in bf16 requires ~140 GB VRAM. Serving 10 tasks: $10 \times 140 \text{ GB} = 1,400 \text{ GB} \rightarrow 18+ \text{ A100 80GB GPUs}$ needed simultaneously.

- Task switching: must reload full model checkpoint per task (10–60 seconds per switch from NVMe SSD)
- New task addition: train new full model (~days of compute), deploy new endpoint, manage N separate serving instances
- Cost estimate: each A100 80GB instance ~\$3/hr on cloud $\rightarrow 18 \text{ GPUs} \times \$3/\text{hr} = \$54/\text{hr}$ just for inference serving
- Memory footprint: $N \times \text{full model size}$ — scales linearly with number of tasks, completely unsustainable at scale
- Engineering overhead: N separate model servers, load balancers, health checks, and monitoring pipelines

This is why multi-adapter architectures (LoRAX, AdapterFusion) exist — sharing one frozen base eliminates this quadratic scaling problem.

LoRAX Architecture — Multi-Adapter Serving Solution

Use Case	Without LoRAX	With LoRAX / PEFT Fusion
Serving 10 tasks	10 model copies (10×140GB=1,400GB VRAM)	1 base + 10 adapters (~36GB total)
Task switching	Reload full checkpoint (10–60 seconds)	Swap adapter weights (microseconds)
New task addition	Train new full model; redeploy endpoint	Train new LoRA adapter; add to pool
Memory footprint	$N \times$ full model size (linear scaling)	$1 \times$ base + $N \times$ ~100–200 MB per adapter
Inference batching	One task type per batch	Multiple tasks batched on same GPU
Estimated cloud cost	18+ A100s ~\$54/hr for 10 tasks	1–2 A100s ~\$3–6/hr for 10 tasks

LoRAX (Predibase): github.com/predibase/lorax | HuggingFace TGI also supports LoRA adapter loading natively.

Multi-Adapter Memory Mathematics

$$\text{Total VRAM (naive)} = N \times \text{model_size} \quad \text{vs} \quad \text{Total VRAM (multi-adapter)} = \text{model_size} + N \times \text{adapter_size}$$

Component	Calculation / Value
Single 70B model (bf16)	$70\text{B} \times 2 \text{ bytes} = 140 \text{ GB VRAM}$
10 full models (naive serving)	$10 \times 140 \text{ GB} = 1,400 \text{ GB}$ (18 A100 80GB GPUs required)
70B base quantised to 4-bit	$70\text{B} \times 0.5 \text{ bytes} = 35 \text{ GB VRAM}$
1 LoRA adapter (r=16, all linear layers)	~100–200 MB (<<1% of base size)
10 adapters simultaneously in memory	$10 \times 150 \text{ MB} = 1.5 \text{ GB additional VRAM}$
Total: quantised base + 10 adapters	$35 \text{ GB} + 1.5 \text{ GB} = 36.5 \text{ GB} \rightarrow$ fits in 1 A100 80GB
VRAM saving factor	$1,400 \text{ GB} \rightarrow 36.5 \text{ GB} = 38\times$ reduction
Cost saving factor	$\sim \$54/\text{hr} \rightarrow \sim \$3/\text{hr} = \sim 18\times$ cost reduction

PEFT in Production — 6-Scenario Decision Matrix

Production Scenario	Recommended Method	Key Reason	Primary Tool
Single task, tight latency SLA	LoRA (merged)	Zero inference overhead after merge	HuggingFace PEFT
Multi-task classification API	Adapters + AdapterFusion	Dynamic task composition from shared base	AdapterHub
Consumer GPU fine-tuning (<24GB)	QLoRA	4-bit base enables single RTX 3090 training	BnB + HF PEFT
Structured output generation	Prefix Tuning	Prompt-level control of generation format	HuggingFace PEFT
Federated/edge deployment	Compacter	Minimal parameter size for transmission	Custom impl.
Research: PEFT comparison study	LoRA as baseline	Best documented, easiest to reproduce	HuggingFace PEFT

AdapterHub vs HuggingFace PEFT — Ecosystem Map

AdapterHub (adapterhub.ml)

- Primary: Adapter Modules, AdapterFusion, Compacter, UniPELT
- Model support: BERT, RoBERTa, XLNet, GPT-2, T5, BART (encoder-heavy)
- Hub: 700+ pre-trained adapters — download and compose
- Install: `pip install adapter-transformers`
- Usage: `model.load_adapter('nli/bert-base/', source='hf')`
- Best for: NLP classification research, BERT-era multi-task settings

HuggingFace PEFT (github.com/huggingface/peft)

- Primary: LoRA, QLoRA, Prefix Tuning, Prompt Tuning, IA³
- Model support: Llama, Mistral, Falcon, Phi, GPT-NeoX (decoder-heavy)
- Integration: Trainer, TRL, Accelerate, bitsandbytes — seamless
- Install: `pip install peft`
- Usage: `get_peft_model(model, LoraConfig(r=16, lora_alpha=32, ...))`
- Best for: generative LLMs, instruction tuning, RLHF/DPO pipelines

Sub-Module 7.2 — Key Takeaways

- 1 Adapter modules insert small bottleneck FFN layers ($d \rightarrow k \rightarrow d$) into each Transformer block. Only adapter parameters are trained — base stays frozen throughout fine-tuning.
- 2 Near-zero initialisation ($W_{up} = 0$) ensures adapters start as identity functions, preserving the SFT checkpoint's behaviour completely from training step 1.
- 3 Bottleneck dimension k is the primary parameter budget lever: $k=64$ gives $\sim 2\%$ params (higher quality); $k=16$ gives $\sim 0.27\%$ (lower overhead).
- 4 Pfeiffer single-adapter (post-FFN only) achieves comparable GLUE quality to Houlsby dual-adapter at $4\times$ fewer parameters — preferred in production.
- 5 AdapterFusion composes N task adapters without retraining any — the key to scalable multi-task BERT serving via shared frozen base.
- 6 Choose LoRA when zero inference overhead is required (merge after training); choose adapters for BERT-era multi-task or when separability is critical.
- 7 LoRAX multi-adapter serving reduces VRAM from $N \times \text{base}$ to $1 \times \text{base} + N \times 100\text{MB}$ — enabling $\sim 38\times$ VRAM reduction and $\sim 18\times$ cost saving for 10-task deployments.

SUB-MODULE 7.3 · INSTRUCTION TUNING

Instruction-Following Datasets, Data Quality & Multi-Task Tuning

The (instruction, input, output) triple · formats · FLAN · LIMA · Self-Instruct pipeline

System Prompts in ChatML — Persona Injection at Inference Time

Key Capability

System prompts in ChatML format allow injecting persona, safety guidelines, and domain constraints at inference time — without retraining the model. This is the primary mechanism for customising deployed models.

Example: Customer Service Bot

- `<|im_start|>system`
- You are a helpful customer service agent for TechCorp. Always be polite. Only discuss TechCorp products. If asked about competitors, redirect to TechCorp features. Never reveal pricing discounts.
- `<|im_end|>`
- → Model behaves as a branded customer agent
- → No fine-tuning needed — just change the system prompt
- → Can update persona without any model retraining

Example: Code Review Assistant

- `<|im_start|>system`
- You are an expert code reviewer specialising in Python and security. Always check for: SQL injection, input validation, and memory leaks. Provide specific line references in your feedback.
- `<|im_end|>`
- → Model becomes a domain-specific expert
- → Swap system prompts to change domain instantly
- → A/B test different personas without model changes

Important: system prompt injection works because the model was trained on data with system prompts. Models trained on Alpaca (no system turn) cannot use this capability.

Major Public Instruction Datasets — Landscape Overview

FLAN (Google, 2021)

1M+ examples across 60+ NLP tasks. Multiple templates per task. Pioneer of multi-task instruction tuning. Still widely used as a component.

Stanford Alpaca (2023)

52K GPT-3.5-generated examples using Self-Instruct. Cost ~\$500 to generate. Sparked the open-source instruction tuning movement.

ShareGPT (Community, 2023)

~90K real ChatGPT conversations shared by users. Multi-turn. High quality but licence issues — check before commercial use.

OpenAssistant / OASST (2023)

161K human-generated messages in tree structure. 35K conversation trees. 43 languages. Apache 2.0. Best human-quality open dataset.

Dolly (Databricks, 2023)

15K examples written by 5,000 Databricks employees. Apache 2.0 — fully commercial. Covers brainstorming, QA, summarisation, classification.

WizardLM (Microsoft, 2023)

196K complex, evolved instructions. Uses Evol-Instruct to progressively increase instruction difficulty. Strong on reasoning tasks.

Public Instruction Datasets — Full Comparison Table

Dataset	Source	Size	Format	Licence	Best For
FLAN	Google (2021)	1M+ (60+ tasks)	Alpaca-like	Research	General instruction following
Stanford Alpaca	Stanford (2023)	52K	Alpaca	Research (LLaMA restricted)	Lightweight SFT prototype
ShareGPT	Community (2023)	~90K convs	ShareGPT	Research (check ToS)	Multi-turn chat capability
OASST	Open Assistant (2023)	161K messages	Tree-structured	Apache 2.0	Human-quality multi-turn, safe
Dolly	Databricks (2023)	15K	Alpaca	Apache 2.0 (✔ commercial)	Commercial-safe SFT training
WizardLM	Microsoft (2023)	196K	Alpaca	Research	Complex reasoning & instructions

Commercial use: always verify the licence of base model AND dataset. Dolly + Mistral-7B is a safe commercial-friendly starting point.

FLAN — The Pioneer Instruction-Tuned Dataset

FLAN (Wei et al., ICLR 2022)

FLAN (Fine-tuned LAnguage Net) fine-tuned a 137B LM on 62 NLP benchmarks, each reformulated as natural language instructions with multiple templates per task. The resulting model showed strong zero-shot performance on held-out tasks.

62 NLP Task Categories

Summarisation, QA, NLI, translation, sentiment, coreference, reading comprehension, closed-book QA, commonsense, dialogue

10 Templates Per Task

Each task expressed in 10 different ways: varying phrasing, adding options, reversing format. This teaches robustness to prompt variation.

Training Scale

Fine-tuned on 137B parameter LaMDA / PaLM. Training data: millions of (instruction, output) pairs across all 62 tasks × 10 templates.

Key Finding

Models tuned on diverse task mixtures generalise significantly better to held-out tasks than models tuned on individual tasks — diversity > volume.

Stanford Alpaca — Lightweight SFT at Scale

Dataset Size & Cost

52,000 instruction-response pairs generated using GPT-3.5-turbo via the Self-Instruct pipeline. Total generation cost: ~\$500. Demonstrated that high-quality instruction data is affordable.

Self-Instruct Pipeline

175 human-written seed tasks → GPT-3.5 generates diverse variations → quality filter (ROUGE similarity, length, harmful content removal) → 52K final examples.

Training Setup

Fine-tuned on LLaMA-7B for 3 epochs on 4× A100 80GB GPUs. Training took ~3 hours. The fine-tuned model showed qualitatively comparable results to ChatGPT on many tasks.

Impact on the Field

Sparked the open-source instruction tuning movement. Led to Vicuna, Koala, Dolly, and dozens of derived models. Proved small models with good data can approach large model quality.

Limitation: Licence

Built on LLaMA 1 which had a research-only licence. The dataset itself is research-only (inherits restrictions). Do not use for commercial products without verifying the base model licence.

Format Detail

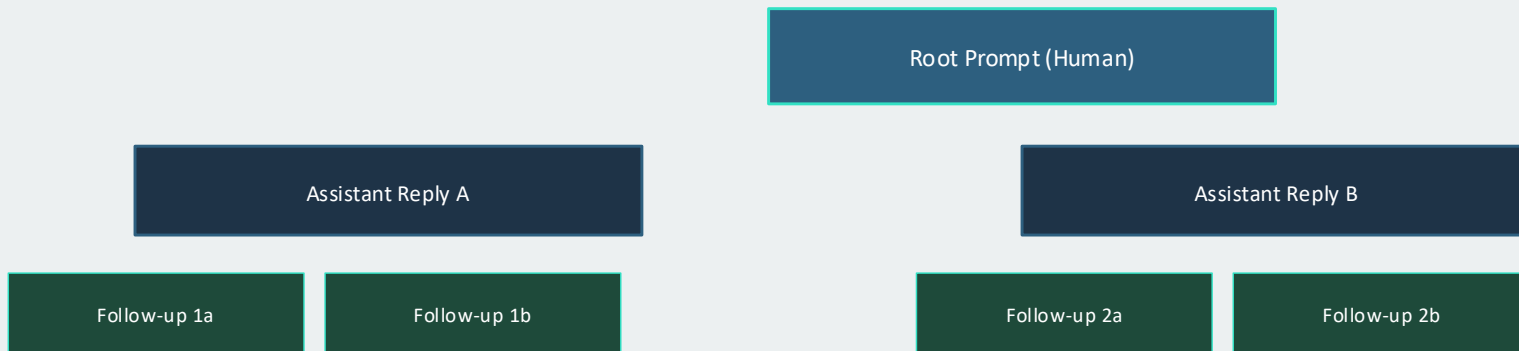
Pure Alpaca format: {instruction, input, output}. Input field is empty for ~40% of examples. Uses the '### Instruction / ### Input / ### Response' prompt template.

OpenAssistant (OASST) — Tree-Structured Human Conversations

OpenAssistant (Köpf et al., NeurIPS 2023)

OASST contains 161,443 messages in 35,108 conversation trees, contributed by 13,500+ volunteers in 43 languages. Apache 2.0 licence — fully open for commercial use. Highest quality open human-generated instruction dataset.

Conversation Tree Structure:



- Each branch represents a different assistant response — enables pairwise preference comparison for RLHF reward model training
- Human annotators rank branches: this creates the (chosen, rejected) pairs needed for DPOTrainer and RewardTrainer
- 43 languages: broadest multilingual coverage of any public instruction dataset
- Use case: SFT training on best-ranked branches; RLHF training on all branch pairs

Dolly — Commercial-Safe Instruction Dataset (Apache 2.0)

Databricks Dolly (2023) — Apache 2.0 Licence

Dolly is a 15K-example instruction dataset written by 5,000 Databricks employees — all human-generated, no GPT output. Apache 2.0 licence means it can be used commercially without restriction, making it one of the safest datasets for production fine-tuning.

Brainstorming (20%)

'Suggest 5 product names for a sleep tracking app'

Open Q&A (25%)

'What causes inflation? Explain simply.'

Closed Q&A (15%)

'What is the capital of Australia?' (with Wikipedia context)

Summarisation (15%)

'Summarise this news article in 2 sentences.'

Information Extraction (15%)

'Extract all dates mentioned in this paragraph.'

Classification (10%)

'Is this customer review positive, negative, or neutral?'

Instruction Tuning — Data Quality Objectives

Objective 1: Quality Pillars

Understand the four pillars of fine-tuning data quality: diversity, correctness, format consistency, and length balance — and how to measure each.

Objective 2: Dataset Construction

Apply the 5-step dataset construction checklist before any fine-tuning run to prevent common data quality failures.

Objective 3: LIMA Finding

Understand why 1,000 curated examples can outperform 52,000 noisy examples — and the implications for data collection strategy.

Objective 4: Multi-Task IT

Build multi-task instruction datasets following the FLAN approach and understand when multi-task IT outperforms single-task fine-tuning.

The 4 Quality Pillars of Fine-Tuning Data

Pillar 1: Diversity

Cover a wide range of tasks, domains, difficulty levels, and response styles. Narrow training data → narrow capability. Measure with entropy over task categories: $H = -\sum p(\text{task}) \log p(\text{task})$.

Pillar 2: Correctness

Ground-truth outputs must be accurate, especially for factual, code, and math tasks. One wrong example can propagate error patterns throughout fine-tuning — model learns to mimic mistakes.

Pillar 3: Format Consistency

Use a single prompt template throughout the entire dataset. Mixing formats confuses the model — it learns competing format patterns rather than the task itself. Validate with an assertion check.

Pillar 4: Length Balance

Mix short (1–2 sentence) and long (paragraph+) responses proportionally. Imbalanced length distribution → biased generation length. Target: p50 < 200 words, p99 < 1,000 words.

Pillar 1: Diversity — How to Measure and Enforce It

Shannon Entropy for Task Diversity

$H = -\sum_i p(\text{task}_i) \times \log_2 p(\text{task}_i)$ where $p(\text{task}_i)$ = fraction of examples from task category i . Higher H = more diverse dataset. Target: $H > 0.8 \times \log_2(N_{\text{categories}})$.

Diversity Sampling Strategy

- Cluster instruction embeddings using k-means (k = number of target categories)
- Sample proportionally from each cluster to ensure balanced task coverage
- Use sentence-transformers to embed: `model.encode(instructions)`
- Set maximum category cap: $< 30\%$ from any single task category
- Add diversity bonus: reward examples from under-represented clusters
- Re-embed and re-cluster after each generation batch for adaptive sampling

Common Diversity Failures

- 50%+ of examples are summarisation tasks $\rightarrow$ model becomes a summariser
- All examples from a single domain (medical, legal) $\rightarrow$ poor generalisation
- Too many 'explain X' tasks $\rightarrow$ model learns one pattern, ignores others
- Uniform difficulty level $\rightarrow$ model misses easy+hard spectrum
- Missing instruction types: no classification, no generation, no extraction
- Fix: compute task category histogram before training, enforce caps

Why Instruction Tuning? Base Model vs Chat Model

Base Model (Pre-trained Only)

- Optimised for: next-token prediction on web text
- Completes text rather than following instructions
- Prompt: 'Summarise this article' → may continue article
- Requires complex few-shot prompt engineering
- Inconsistent behaviour — depends heavily on prompt format
- No awareness of conversation or instruction-response structure

Instruction-Tuned Model (SFT applied)

- Optimised for: following natural language instructions
- Understands task descriptions and responds appropriately
- Prompt: 'Summarise this article' → returns a concise summary
- Works with simple, natural language prompts
- Consistent instruction-following behaviour across tasks
- Trained on (instruction, input, output) triples of diverse tasks

Instruction tuning is the bridge between raw pre-trained capability and deployed assistant behaviour. It costs a fraction of pre-training but dramatically improves usability.

Pillar 2: Correctness — Why One Bad Example Propagates

The Error Propagation Problem

A single incorrect output in the training set teaches the model a wrong pattern. Because fine-tuning optimises to minimise loss on all examples equally, incorrect examples receive the same gradient weight as correct ones — actively training the model to be wrong.

Training Example

'Q: What is 15% of 80? A: 15' (WRONG — correct is 12)

Gradient Signal

Model loss minimised on this example → model parameters updated to produce '15' for similar calculations

Inference Effect

Model now calculates 15% of ANY number as 15 — error pattern generalises across similar prompts

Scale Effect

If 5% of 1,000 examples are wrong → 50 bad examples competing with 950 good ones → measurable accuracy degradation

Mitigation: LLM-as-judge scoring (GPT-4 scores each output 1–5, keep ≥4); random spot-check 50 examples manually; unit-test code outputs by execution

Pillar 3: Format Consistency — The Competing Pattern Problem

Why Mixed Formats Are Harmful

When the training dataset contains multiple prompt formats (some Alpaca, some ChatML, some custom), the model must simultaneously learn: (a) what the task is and (b) which format applies. These competing objectives hurt task learning.

Consistent Format (Correct)

- ALL examples use identical ChatML template
- `<|im_start|>system ... <|im_end|>`
- `<|im_start|>user ... <|im_end|>`
- `<|im_start|>assistant ... <|im_end|>`
- Model learns ONE pattern → full gradient capacity used for task learning
- Validation: assert all examples parse correctly with single tokenizer call

Mixed Formats (Harmful)

- Example 1: `### Instruction: ... ### Response:`
- Example 2: `<|im_start|>user ... <|im_end|>`
- Example 3: `Human: ... Assistant:`
- Model learns 3 competing patterns → confusion at inference
- Often manifests as: wrong role prefix, repeated template tokens in output
- Fix: run format normalisation script before any training run

Enforcement: `assert all(example['format'] == 'chatml' for example in dataset)`. Fail fast before training starts.

Pillar 4: Length Balance — Distribution and Targets

Length Imbalance Effect

If 90% of training outputs are 1–2 sentences and 10% are paragraphs, the model learns a strong prior toward short responses. When asked for a detailed explanation, it will truncate prematurely. The reverse also applies — padding-dominated data → verbose outputs.

Length Metric	Target Value	Why It Matters	How to Compute
p50 (median) output length	< 200 words	Ensures typical responses are concise	<code>np.percentile(lengths, 50)</code>
p99 output length	< 1,000 words	Prevents extreme outliers from dominating	<code>np.percentile(lengths, 99)</code>
p1 (minimum)	≥ 10 words	Removes trivially short/empty outputs	<code>np.percentile(lengths, 1)</code>
Short responses (<50 words)	20–35% of dataset	Balance with medium/long responses	<code>sum(l < 50 for l in lengths)</code>
Long responses (>300 words)	10–20% of dataset	Ensure model can produce detail when needed	<code>sum(l > 300 for l in lengths)</code>
Zero-length outputs	0 examples	Hard filter: remove all empty responses	<code>len([l for l in lengths if l == 0])</code>

Measuring Diversity — Shannon Entropy Calculation Example

```
from collections import Counter
import math

# Example: 1000-example dataset with task category labels
task_labels = ["summarisation"] * 300 + ["qa"] * 250 + ["translation"] * 150 +
              ["classification"] * 150 + ["code"] * 100 + ["extraction"] * 50

def compute_diversity_entropy(labels):
    n = len(labels)
    counts = Counter(labels)
    entropy = -sum((c/n) * math.log2(c/n) for c in counts.values())
    max_entropy = math.log2(len(counts)) # Perfect uniform distribution
    normalized = entropy / max_entropy # 0 = single category, 1 = perfect uniform
    return {"entropy": round(entropy, 3), "max_entropy": round(max_entropy, 3),
            "normalized": round(normalized, 3), "distribution": dict(counts)}

result = compute_diversity_entropy(task_labels)
print(f"Entropy: {result['entropy']} / {result['max_entropy']} (max)")
print(f"Normalized diversity: {result['normalized']:.1%}") # 0.953 = 95.3% of max
# → Action: if normalized < 0.7, add more examples from under-represented categories
# → Enforce: max single category = 30%
print(f"Max category fraction: {max(result['distribution'].values())/len(task_labels):.1%}")
```

Target: normalized entropy > 0.80. If summarisation > 30%, cap it and oversample other categories via diversity sampling.

Quality-First Data Selection — The LIMA Approach

LIMA Principle (Zhou et al., NeurIPS 2023)

1,000 carefully curated, diverse, high-quality examples rival or outperform 52,000 GPT-generated Alpaca examples on human preference evaluations. Almost all knowledge is in the pre-trained model — fine-tuning teaches the format to express it.

- Step 1 — Manual curation: review every example for factual accuracy, clarity, and appropriate length; discard borderline cases
- Step 2 — Diversity sampling: cluster instruction embeddings, sample from under-represented clusters to maximize task coverage
- Step 3 — Perplexity-based filtering: remove examples where base model perplexity < 15 (already knows them) or > 200 (incoherent)
- Step 4 — LLM-as-judge scoring: use GPT-4 to score each (instruction, output) pair on a 1–5 rubric; keep only scores ≥ 4
- Step 5 — Length balance: compute histogram, ensure $p_{50} < 200$ words; remove both zero-length and extreme outliers
- Step 6 — Format validation: assert all examples use the same ChatML template; fail fast before any training run

Quantity-First Data Collection — Web-Scale Pipeline

When Quantity Matters

For pre-training and continued pre-training on domain corpora, raw scale is essential — models need exposure to billions of tokens. Web-scale pipelines automate quality filtering across massive datasets.

- MinHash deduplication: $O(n)$ near-duplicate detection using LSH with Jaccard threshold 0.8–0.9 — fastest first step
- Perplexity thresholding: use a reference LM (TinyLlama/KenLM) to score each document — remove top/bottom 10%
- Toxicity classifier: Perspective API or detoxify library to flag harmful content — remove examples above threshold
- Language identification: fastText lid.176 to filter non-target language examples from multi-language crawls
- URL and domain filtering: remove known spam/adult/low-quality domains using blocklists (UT-1, LDNOOBW)
- Quality classifier: train a text classifier on Wikipedia vs random web (CCNet approach) — keep Wikipedia-like content

Quality vs Quantity — Side-by-Side Decision Guide

Dimension	Quality-First (LIMA-style)	Quantity-First (Web-scale)
Dataset size	1,000–10,000 examples	100K–100M+ examples
Human review	Every example reviewed	Statistical sampling only
Example accuracy	Manually verified	Automated heuristic filters
Training use case	Instruction tuning / SFT	Pre-training / domain adaptation
Cost per example	High (\$5–50 human annotation)	Low (\$0.001 automated pipeline)
Performance on benchmarks	Often outperforms larger noisy sets	Good baseline, benefits from quality layer
Time to produce	Weeks (careful curation)	Hours/days (automated pipeline)
Risk of noise	Very low	5–10% noise even after filtering

LLM-as-Judge for Automated Data Quality Scoring

```
import openai, json
client = openai.OpenAI()

QUALITY_PROMPT = """Rate this instruction-following example on a 1-5 scale.
Instruction: {instruction}
Response: {response}
Rate on: (1) Factual accuracy (2) Instruction following (3) Clarity (4) Helpfulness
Return ONLY JSON: {"accuracy": N, "following": N, "clarity": N, "helpful": N, "overall": N}"""

def score_example(ex):
    r = client.chat.completions.create(model="gpt-4o-mini",
        messages=[{"role": "user", "content": QUALITY_PROMPT.format(
            instruction=ex["instruction"], response=ex["output"])}],
        response_format={"type": "json_object"})
    return json.loads(r.choices[0].message.content)

# Filter: keep only high-quality examples (overall >= 4)
high_quality = [ex for ex in dataset if score_example(ex)["overall"] >= 4]
print(f"Kept {len(high_quality)}/{len(dataset)} examples ({len(high_quality)/len(dataset):.1%})")
```

Cost: gpt-4o-mini ~\$0.001/example. Scoring 10K examples costs ~\$10. Far cheaper than human annotation at \$0.50–5.00/example.

Perplexity-Based Filtering — Identifying Low-Value Examples

Perplexity as a Quality Signal

Perplexity measures how surprised a reference language model is by a text. Low perplexity = model already knows this well (trivial, low training value). High perplexity = model finds text incoherent or out-of-distribution. Both extremes should be filtered.

Very Low Perplexity (< 15)

Base model already handles this well. Fine-tuning signal provides no additional learning — wasted compute. Example: 'What is 2+2? Answer: 4'

Good Range (15–200)

Model struggles but can learn — productive training signal. Examples require real understanding: complex reasoning, domain knowledge, multi-step tasks.

Very High Perplexity (> 200)

Text is likely incoherent, severely out-of-distribution, or garbled. Including these examples actively degrades model quality. Filter aggressively.

MinHash Deduplication — How Near-Duplicate Detection Works

MinHash + Locality-Sensitive Hashing (LSH)

MinHash efficiently estimates the Jaccard similarity between documents (fraction of shared n-grams) without comparing all pairs directly. LSH groups similar documents into the same bucket — only bucket members are compared, making it $O(n)$ rather than $O(n^2)$.



- Computational complexity: $O(n)$ vs $O(n^2)$ brute force — 1M examples deduped in seconds instead of hours
- datasketch library: `pip install datasketch` → `MinHash(num_perm=128)` for 1% error rate at Jaccard 0.8
- Threshold tuning: 0.9 = only near-identical; 0.7 = removes paraphrased duplicates (more aggressive)

The (Instruction, Input, Output) Triple — Dataset Anatomy

The Basic Training Unit

Each instruction tuning example consists of three fields: (1) Instruction — the task description, (2) Input — optional context or data to process, (3) Output — the desired model response. The model learns to follow the instruction given any input.

Instruction

Translate the following English sentence to French.

Specifies the task the model must perform. Should be clear, specific, and actionable.

Input (optional)

The quick brown fox jumps over the lazy dog.

Provides the specific content to process. Can be empty string "" for knowledge tasks with no input context.

Output

Le renard brun rapide saute par-dessus le chien paresseux.

The ground-truth response the model should produce. Quality of this field directly determines fine-tuning quality.

Dataset Construction Checklist — 5 Steps Before Training

1. Format Validation

```
assert all(ex["format"] == "chatml" for ex in dataset)
assert all(len(ex["output"]) > 0 for ex in dataset)
```

Fail fast: if format validation fails, fix the preprocessing pipeline before running any training.

2. Near-Deduplication

```
python dedup.py --threshold 0.9 --input data.jsonl --output
data_dedup.jsonl
```

Run before other filters. Duplicates waste compute and introduce repetition bias.

3. Length Distribution

```
lengths = [len(ex['output'].split()) for ex in dataset]
# Target: p50 < 200, p99 < 1000, no zero-length
```

Compute histogram, check percentiles, filter outliers.

4. Correctness Spot-Check

```
# Sample 50 random examples; manually review for accuracy
```

Sample 5% (min 50 examples) for manual review. One reviewer per domain area.

5. Task Diversity

```
cats = Counter(ex['task_category'] for ex in dataset)
assert max(cats.values())/len(dataset) < 0.30 # No single category > 30%
```

Enforce diversity cap. If any category exceeds 30%, downsample it.

Dataset Validation — Full Checklist Code

```
import json, math
from collections import Counter
from datasketch import MinHash, MinHashLSH

def validate_dataset(path, max_cat_fraction=0.30, min_len=10, max_len=1000):
    with open(path) as f:
        data = [json.loads(l) for l in f]

    # 1. Format check
    assert all("output" in ex and len(ex["output"]) > 0 for ex in data), "Empty outputs found"

    # 2. Near-dedup
    lsh = MinHashLSH(threshold=0.9, num_perm=128)
    deduped = []
    for i, ex in enumerate(data):
        mh = MinHash(num_perm=128)
        for w in ex["instruction"].lower().split(): mh.update(w.encode())
        if not lsh.query(mh):
            lsh.insert(f"ex_{i}", mh); deduped.append(ex)
    print(f"After dedup: {len(deduped)}/{len(data)} ({{len(deduped)/len(data):.1%}})")

    # 3. Length check
    lens = [len(ex["output"].split()) for ex in deduped]
    import numpy as np
    p50, p99 = np.percentile(lens, 50), np.percentile(lens, 99)
    filtered = [ex for ex, l in zip(deduped, lens) if min_len <= l <= max_len]
    print(f"p50={{p50:.0f}} words | p99={{p99:.0f}} words | After filter: {len(filtered)}")

    # 4. Diversity check
    if "task_category" in filtered[0]:
        cats = Counter(ex["task_category"] for ex in filtered)
        max_frac = max(cats.values()) / len(filtered)
        assert max_frac < max_cat_fraction, f"Category imbalance: {max_frac:.1%} > {max_cat_fraction:.1%}"
    return filtered
```

Run this validation function before every training run. Treat it like unit tests for your data pipeline.

Length Distribution Analysis — Histogram Computation

```
import numpy as np
from collections import Counter

# Compute output length distribution
lengths = [len(ex['output'].split()) for ex in dataset]

# Percentile analysis
p1, p10, p50, p90, p99 = np.percentile(lengths, [1, 10, 50, 90, 99])
print(f"p1={p1:.0f} p10={p10:.0f} p50={p50:.0f} p90={p90:.0f} p99={p99:.0f}")

# Bucket analysis
short = sum(1 < 50 for l in lengths) # Target: 20-35%
medium = sum(50 <= l < 200 for l in lengths) # Target: 40-50%
long = sum(200 <= l < 500 for l in lengths) # Target: 15-25%
vlong = sum(l >= 500 for l in lengths) # Target: <10%
n = len(lengths)
print(f"Short (<50): {short/n:.1%} Medium: {medium/n:.1%} Long: {long/n:.1%} VLong: {vlong/n:.1%}")

# Red flags to fix:
# • p50 > 400: dataset biased toward verbose outputs → model will pad
# • p1 == 0: empty outputs exist → add min_len filter of 10 words
# • vlong > 20%: too many very long examples → truncate at 1000 tokens max
# • short > 60%: too many one-liners → add more detailed response examples

# Fix: filter out extreme outliers
filtered = [ex for ex, l in zip(dataset, lengths) if 10 <= l <= 1000]
print(f"Filtered: {len(filtered)}/{len(dataset)} ({len(filtered)/len(dataset):.1%}")
```

Compute this histogram before every training run. Biased length distribution is the most common cause of unexpected output length patterns in fine-tuned models.

Multi-Task Instruction Tuning — Core Definition

Definition

Multi-task instruction tuning trains the model simultaneously on a mixture of many different NLP tasks, each reformulated as a natural language instruction. The model learns to generalise across tasks — improving zero-shot performance on unseen tasks at inference time.

- FLAN (Wei et al., 2022): 62 NLP tasks × 10 instruction templates each = the model learns task-general instruction following
- The diversity of tasks is more important than the volume of any single task — breadth beats depth for zero-shot generalisation
- Key finding: models tuned on task mixtures generalise significantly better to held-out tasks than models tuned on any single task
- Template variety per task teaches robustness to prompt phrasing variation — the model stops being sensitive to exact wording
- Multi-task IT is the approach behind FLAN, Flan-T5, T0, and all modern general-purpose instruction-tuned models

Why Task Diversity > Volume — The Generalisation Evidence

FLAN Ablation Finding (Wei et al., 2022)

Adding more tasks to the training mixture consistently improves zero-shot performance on held-out task clusters — even when total training tokens are held constant. Volume within a single task shows diminishing returns beyond ~100 examples per task.

Training Setup	Held-Out Task Performance	Interpretation
1 task (QA only)	Low — overfits to QA format	Cannot generalise to NLI, summarisation
10 tasks (varied)	Medium — learns some general patterns	Partial instruction following capability
62 tasks (FLAN)	High — strong zero-shot generalisation	Learns task-general instruction semantics
62 tasks × 10 templates	Highest — robust to prompt variation	Template diversity prevents overfitting to phrasing
Same 62 tasks, 10× more data	Marginal gain over 62 tasks	Volume has diminishing returns vs diversity

Practical implication: when building instruction datasets, prioritise adding new task categories over adding more examples of existing tasks.

Template Variety — Teaching Prompt Robustness

The Problem: Prompt Sensitivity

Without template variety, a fine-tuned model learns to respond only to the specific phrasing used in training. Minor rephrasing of the instruction at inference time causes dramatic performance drops.

Example: Same NLI task expressed in 4 different templates

Template 1 (Direct)

Does the following sentence entail, contradict, or neither? Sentence: [X]. Hypothesis: [Y]. Answer:

Template 2 (Q&A)

Given that [X] is true, can we conclude that [Y]? Choose: yes/no/maybe.

Template 3 (Reasoning)

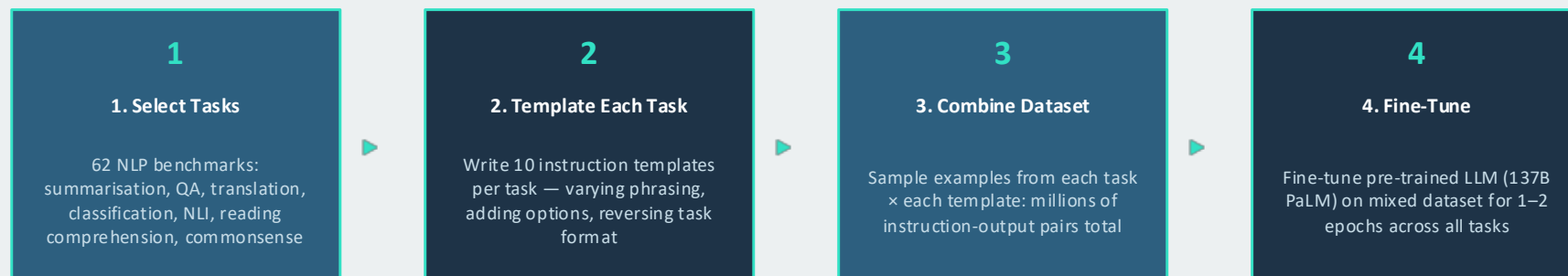
Premise: [X]. Does this premise support the claim [Y]? (supports / contradicts / no relation)

Template 4 (Reversed)

[Y] — does this follow from [X]? Select: Entailment, Contradiction, or Neutral.

Model trained on all 4 templates learns NLI itself — not a specific format. Model trained on template 1 only fails on templates 2–4.

How FLAN Was Built — 4-Stage Process



Scale: FLAN-T5 (2022) extended this to 1,836 tasks with chain-of-thought. Flan-PaLM 2 used 1,000+ tasks. The pattern always holds: more diverse tasks → stronger generalisation.

FLAN Generation	Tasks	Templates per Task	Base Model	Key Contribution
FLAN (2021)	62	10	LaMDA 137B	Zero-shot task generalisation demonstrated
FLAN-T5 (2022)	1,836	varies	T5 11B	Chain-of-thought + scaled task mixture
Flan-PaLM 2 (2023)	1,000+	10+	PaLM 2 540B	SOTA on multiple reasoning benchmarks

FLAN Task Categories — Breadth of Coverage

Summarisation

Examples:

CNN/DailyMail, XSum, SAMSum

Question Answering

Examples:

Natural Questions, TriviaQA, SQuAD

Sentiment Analysis

Examples:

SST-2, IMDB, Yelp reviews

Natural Language Inference

Examples:

SNLI, MultiNLI, ANLI

Translation

Examples:

WMT14 EN-DE, EN-FR

Coreference Resolution

Examples:

WSC273, DPR, Winogrande

Reading Comprehension

Examples:

BoolQ, RACE, QuAC

Commonsense Reasoning

Examples:

HellaSwag, PIQA, ARC

Total: 62 tasks spanning 12 task clusters in FLAN original. FLAN-T5 expanded to 1,836 tasks. The key is horizontal breadth — covering all major NLP task types.

Writing Instruction Templates — Before & After

Task: Natural Language Inference (NLI) — Converting a Benchmark to Instruction Format

Raw NLI benchmarks provide (premise, hypothesis, label) triples. To use for instruction tuning, each must be reformulated as a natural language instruction-response pair. Crucially, the same task gets 10 different instruction phrasings.

Raw NLI Benchmark Format

- premise: 'The dog barked at the mailman'
- hypothesis: 'The dog was quiet'
- label: 2 (contradiction)
- This is machine-readable but NOT an instruction.
- No template → model gets no guidance on what to do.
- Cannot be used directly for instruction fine-tuning.

Instruction Template Variants

- T1: 'Does [premise] entail [hyp]? Yes/No/Maybe'
- T2: 'Given [premise], can we conclude [hyp]? Choose.'
- T3: 'Relationship between [premise] and [hyp]?'
- T4: '[Hyp] — true given [premise]?
Entailment/Contradiction/Neutral'
- T5: 'If [premise] is true, is [hyp] also true?'
- → Same training example × 5 templates = 5 training pairs

Template Reversal — Doubling Data with Inverted Tasks

Template Reversal Technique

For many NLP tasks, the task can be reversed: given the output, generate the input. This doubles the training data at zero annotation cost and teaches the model both directions of the task relationship.

Original Task	Original Template	Reversed Task	Reversed Template
Sentiment → Label	'What is the sentiment? [text]'	Label → Sentiment	'Give a [positive] review: ...'
Summary → Article	'Summarise: [article]'	Article → Summary	'What article has this summary: [sum]'
Question → Answer	'Answer: [question]'	Answer → Question	'What question does this answer: [ans]'
Premise → Conclusion	'Does [prem] entail [hyp]?'	Conclusion → Premise	'Write a premise that entails: [hyp]'
Sentence → Translation	'Translate to French: [en]'	Translation → Source	'Translate from French: [fr]'

Reversal insight: training on both directions makes the model understand the task structure deeply — not just pattern-matching surface forms.

Format Shapes Behaviour — The Core Insight

Critical Observation

A model fine-tuned on Alpaca single-turn format behaves differently from one trained on ShareGPT multi-turn format — even when using the exact same base model. Format is not just syntax; it determines what behaviour the model learns.

Single-Turn Format (Alpaca)

- Each example is an independent (instruction, output) pair
- Model learns: 'given a task description, produce an answer'
- Does NOT learn: how to maintain conversation context
- Use when: building a task-following model (not a chatbot)
- Strength: simpler, very effective for specific task fine-tuning
- Example: Stanford Alpaca, FLAN, WizardLM datasets

Multi-Turn Format (ShareGPT / ChatML)

- Examples are conversation threads with multiple exchanges
- Model learns: 'maintain context across turns, respond to follow-ups'
- DOES learn: conversation memory, clarification, context tracking
- Use when: building an assistant or chatbot product
- Strength: enables human-like conversational interaction
- Example: ShareGPT, OASST, ChatML-formatted datasets

System prompts in ChatML inject persona/safety at inference: `<|im_start|>system You are a helpful assistant.<|im_end|>`

Multi-Task IT vs Single-Task Fine-Tuning — When to Use Each

Scenario	Multi-Task IT	Single-Task FT
Goal: general-purpose assistant	✅ Best choice — broad task coverage	❌ Narrow capability, poor generalisation
Goal: one specific domain (e.g. medical NER)	❌ Wasteful — dilutes task-specific signal	✅ Best choice — focused gradient signal
Zero-shot generalisation to new tasks	✅ Multi-task gives strong zero-shot	❌ Overfits to training task format
Data: many task types available	✅ Leverage diversity across tasks	N/A — single task only
Data: single domain corpus only	❌ Not enough task variety for IT	✅ Focused continued pre-training
Compute budget: limited	High (N tasks × M templates)	Low (single focused dataset)
Need: production chatbot	✅ ChatML multi-task + RLHF alignment	❌ Insufficient for assistant behaviour

Zero-Shot Generalisation — The Key Benefit of Multi-Task IT

Zero-Shot Generalisation Defined

A model is zero-shot on a task if it was never fine-tuned on examples from that specific task. Multi-task IT dramatically improves zero-shot performance by teaching the model to understand instruction semantics generally — not just task-specific patterns.

Without Multi-Task IT

- Model sees instruction: 'Classify this email as spam or not spam'
- Has no fine-tuning on email classification
- Attempts to continue text rather than classify
- Performance $\approx$ random baseline ($\sim 50\%$ on binary tasks)
- Even reasonable phrasing fails — no instruction understanding
- Each new task requires task-specific fine-tuning data

With Multi-Task IT (FLAN)

- Model sees same instruction as above
- Recognises classification task pattern from 62 tasks
- Produces: 'Not spam' correctly in zero-shot setting
- Performance $\gg$ baseline on 25 of 25 held-out tasks
- Works with paraphrased instructions (template variety trained)
- New tasks work out-of-the-box — no additional data needed

The LIMA Finding — Less Is More for Alignment

LIMA (Zhou et al., NeurIPS 2023)

A model fine-tuned on just 1,000 carefully selected, diverse, high-quality examples (LIMA dataset) matches or outperforms models fine-tuned on 52,000 GPT-generated Alpaca examples — on human preference evaluations.

- Interpretation: almost all of the knowledge needed for instruction following is already present in the pre-trained model. Fine-tuning teaches the model the surface format to express that knowledge — not the knowledge itself
- Alignment Tax: large noisy datasets can actually hurt performance on reasoning tasks by flooding the model with low-quality patterns that compete with the task signal
- Quality Primacy: invest heavily in dataset curation, not just volume. The LIMA 1K examples were hand-selected by researchers with deep knowledge of the task
- Practical implication: for most fine-tuning projects, curating 500–2,000 high-quality examples is more valuable than collecting 50K automated examples

The Surface Format Hypothesis — Why LIMA Works

Core Hypothesis

Pre-training on trillion-token web corpora encodes virtually all world knowledge. Fine-tuning does not teach new knowledge — it teaches the model what format and style to use when expressing that already-encoded knowledge in response to instructions.

What Pre-Training Learns

- Factual knowledge from Wikipedia, books, papers
- Mathematical reasoning from educational content
- Coding patterns from GitHub, Stack Overflow
- Language patterns across 100+ languages
- Common sense from diverse text domains
- Scientific concepts from arXiv, PubMed

What Fine-Tuning Adds

- How to recognise instruction format and intent
- When to respond helpfully vs decline appropriately
- Output structure: lists vs prose vs code blocks
- Appropriate response length for different tasks
- Persona-appropriate tone and register
- When to ask for clarification vs proceed directly

Evidence: LIMA 1K examples → same quality as 52K Alpaca. Both models use the same 65B LLaMA base. Only the fine-tuning data quality differs. The base model had all the knowledge — LIMA just taught it how to express it.

Alignment Tax — When More Data Hurts

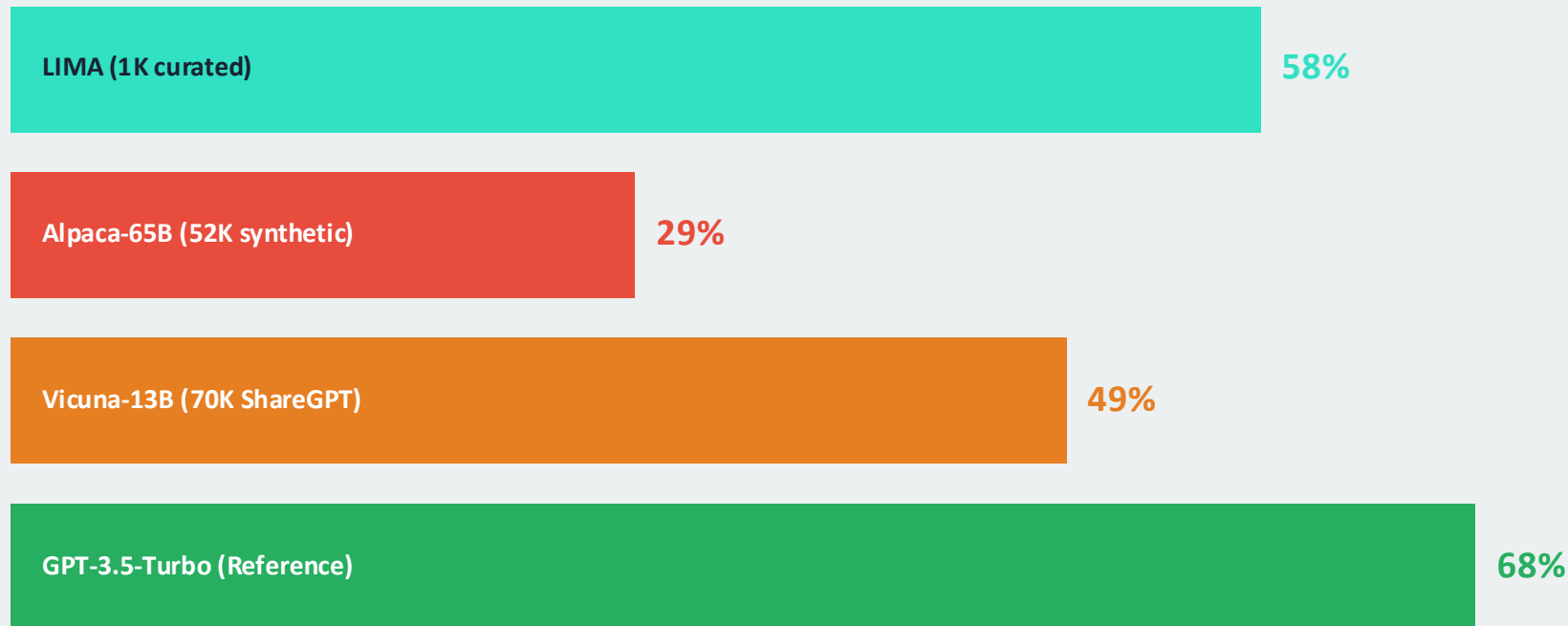
Alignment Tax Defined

When fine-tuning on noisy data, the model must simultaneously learn good patterns AND unlearn bad patterns. The gradient signal from noisy examples actively competes with the signal from good examples — reducing overall performance on reasoning and factual tasks.

Base Model (Pre-trained)	MMLU: 68.4% GSM8K: 57.2%	Strong reasoning from pre-training signal
+ 52K Noisy Alpaca SFT	MMLU: 64.1% GSM8K: 51.3%	Alignment tax: ~4% drop in reasoning accuracy
+ RLHF (PPO on noisy SFT)	MMLU: 61.8% GSM8K: 48.9%	Compound alignment tax: further ~2% drop
+ 1K LIMA-quality SFT	MMLU: 67.9% GSM8K: 56.1%	Minimal alignment tax: only ~0.5% drop from base

LIMA Human Preference Results — Win Rate Comparison

Human Preference Win Rate (% of evaluations where this model was preferred, head-to-head vs fixed reference):



- LIMA (58%) beats Alpaca (29%) by 2× despite using only 1.9% of the training data
- Vicuna (49%) uses ShareGPT multi-turn format — better than Alpaca but uses 70K examples
- GPT-3.5 (68%) sets the human-level reference ceiling for these evaluations

Data Selection Strategies — Four Practical Methods

Perplexity-Based Filtering

Remove examples where base model perplexity < 15 (trivial) or > 200 (incoherent). Use TinyLlama as a fast reference model. Focus training signal on the productive 15–200 range.

Diversity Sampling via k-Means

Embed all instructions with sentence-transformers. Cluster with k-means (k = target categories). Sample proportionally from each cluster to ensure balanced task coverage across the dataset.

LLM-as-Judge Quality Scoring

Use GPT-4o-mini to score each (instruction, response) pair on a 1–5 rubric covering accuracy, following, clarity, and helpfulness. Keep only scores ≥ 4 . Cost: $\sim \$0.001/\text{example}$.

Deduplication First (Always)

Always run MinHash deduplication before any other filter. Near-duplicate examples waste compute and introduce repetition bias. Jaccard threshold 0.9 for instruction pairs.

Lab 7.4 Roadmap — Building a Production Instruction Dataset



Stage	Output	Tool	Estimated Time
Choose Domain	Domain specification doc	Manual decision	30 min
Generate Pairs	500–2K raw examples	GPT-4o API	2–4 hours
Quality Filter	300–1K filtered examples	datasketch + GPT-4o-mini	1–2 hours
Convert Format	JSONL in ChatML format	Python script	30 min
Upload to Hub	Public/private dataset on HF	datasets library	15 min

Lab: Synthetic Data Generation with GPT-4o (Self-Instruct Pipeline)

```
import openai, json, random
client = openai.OpenAI()

SEED_TASKS = [
    "Explain the difference between list and tuple in Python",
    "Write a function to check if a string is a palindrome",
    "What is the time complexity of binary search?",
]

def generate_instruction_pair(seed_task):
    response = client.chat.completions.create(
        model="gpt-4o",
        messages=[{"role": "user", "content": f"""Generate a diverse, high-quality instruction-response pair
similar in style to: '{seed_task}'
Return ONLY valid JSON (no markdown):
{{"instruction": "...", "input": "...", "output": "..."}}
Requirements:
- instruction: clear, specific, actionable Python/coding task
- input: relevant code snippet or context (empty string "" if not needed)
- output: accurate, well-explained, 50-300 words"""}],
        response_format={"type": "json_object"}
    )
    return json.loads(response.choices[0].message.content)

# Generate 500 examples with progress
dataset = []
for i in range(500):
    seed = random.choice(SEED_TASKS)
    try:
        ex = generate_instruction_pair(seed)
        ex["source"] = "gpt4o_selfinstruct"; ex["seed"] = seed
        dataset.append(ex)
        if i % 50 == 0: print(f"Generated {i+1}/500")
    except Exception as e:
        print(f"Skipped {i}: {e}")

with open("raw_dataset.jsonl", "w") as f:
    for ex in dataset: f.write(json.dumps(ex) + "\n")
```

Self-Instruct pattern: small seed set (175 examples) → GPT-4o generates diverse variations → quality filter → final dataset. This is how Stanford Alpaca was built.

Lab: MinHash Deduplication + ChatML Format Conversion

```

from datasketch import MinHash, MinHashLSH
import json

with open("raw_dataset.jsonl") as f:
    dataset = [json.loads(l) for l in f]

# Step 1: MinHash near-deduplication (Jaccard threshold 0.9)
lsh = MinHashLSH(threshold=0.9, num_perm=128)
deduplicated = []
for i, example in enumerate(dataset):
    mh = MinHash(num_perm=128)
    for word in example["instruction"].lower().split():
        mh.update(word.encode('utf8'))
    if not lsh.query(mh): # No near-duplicates found in LSH index
        lsh.insert(f"ex_{i}", mh)
        deduplicated.append(example)

print(f"Before dedup: {len(dataset)} | After: {len(deduplicated)} | Removed: {len(dataset)-len(deduplicated)}")

# Step 2: Length filter (remove trivially short or excessively long outputs)
filtered = [ex for ex in deduplicated if 30 <= len(ex["output"].split()) <= 500]
print(f"After length filter: {len(filtered)}")

# Step 3: Convert to ChatML format for training
def to_chatml(ex):
    user_content = ex["instruction"]
    if ex.get("input", "").strip():
        user_content += "\n\n" + ex["input"]
    return {"conversations": [
        {"from": "system", "value": "You are a helpful Python programming assistant."},
        {"from": "human", "value": user_content},
        {"from": "assistant", "value": ex["output"]}
    ]}

with open("dataset_chatml.jsonl", "w") as f:
    for ex in filtered:
        f.write(json.dumps(to_chatml(ex)) + "\n")
print(f"Saved {len(filtered)} examples in ChatML format")

```

datasketch MinHashLSH is $O(n)$ — deduplicates 100K+ examples in seconds. Threshold 0.9 catches near-identical examples; lower threshold (0.7) catches more paraphrases.

Single-Turn vs Multi-Turn Dataset Capability

Capability	Alpaca (Single-Turn)	ShareGPT (Multi-Turn)	OASST (Tree-Structured)
Task instruction following	✅ Excellent	✅ Good	✅ Good
Multi-turn conversation	❌ No — single exchange only	✅ Yes — full conversation threads	✅ Yes — branching conversations
Context retention	❌ No context across turns	✅ Context maintained across turns	✅ Context with tree branches
Persona/system prompt	Limited (no system turn)	✅ ChatML system prompt support	Partial
Training data size	52K (Stanford Alpaca)	~90K conversations	161K messages (35K trees)
Best use case	Specific task SFT	General-purpose chatbot	Safety-focused assistant
Commercial licence	Research (LLAMA restricted)	Research (restricted)	Apache 2.0 (open)

Lab: LLM-as-Judge Quality Filtering

```
import openai, json
client = openai.OpenAI()

JUDGE_PROMPT = """You are an expert evaluator of AI instruction-following data quality.
Rate this example on a 1-5 scale for each dimension.

Instruction: {instruction}
Response: {response}

Dimensions:
1. accuracy: Is the response factually correct? (code must work, facts must be right)
2. following: Does it directly address the instruction?
3. clarity: Is it clear and well-structured?
4. completeness: Is it thorough without padding?

Return ONLY valid JSON:
{{"accuracy": N, "following": N, "clarity": N, "completeness": N, "overall": N}}"""

def score_example(ex):
    r = client.chat.completions.create(
        model="gpt-4o-mini", # Cheap: ~$0.001/example
        messages=[{"role": "user", "content": JUDGE_PROMPT.format(
            instruction=ex["instruction"], response=ex["output"]}],
        response_format={"type": "json_object"})
    return json.loads(r.choices[0].message.content)

# Score all examples and filter (keep overall >= 4)
scored = []
for ex in filtered:
    scores = score_example(ex)
    ex["quality_scores"] = scores
    if scores["overall"] >= 4:
        scored.append(ex)

print(f"High-quality examples: {len(scored)}/{len(filtered)} ({len(scored)/len(filtered):.1%}")
# Typical result: 60-80% of filtered examples pass the quality gate at threshold 4
```

Cost estimate: 1,000 examples × \$0.001 = \$1. Much cheaper than human annotation at \$0.50–5.00/example. Effective at catching factual errors and instruction mismatches.

Lab: Upload to HuggingFace Hub with Dataset Card

```
from datasets import Dataset, DatasetDict
import json

# Load the filtered, scored, ChatML-formatted dataset
with open("dataset_chatml.jsonl") as f:
    data = [json.loads(l) for l in f]

# Create HuggingFace Dataset object
ds = Dataset.from_list(data)

# Split into train/validation (90/10)
splits = ds.train_test_split(test_size=0.1, seed=42)
ds_dict = DatasetDict({"train": splits["train"], "validation": splits["test"]})

# Push to HuggingFace Hub (requires: huggingface-cli login)
ds_dict.push_to_hub(
    "your-username/python-instruction-dataset-chatml",
    private=True # Set False for public sharing
)
print(f"Uploaded {len(ds_dict['train'])} train + {len(ds_dict['validation'])} val examples")
print("Dataset available at: https://huggingface.co/datasets/your-username/python-instruction-dataset-chatml")

# Dataset card (README.md): describe source, licence, filtering steps, format
# Add it via the HuggingFace Hub UI or programmatically:
# from huggingface_hub import upload_file
# upload_file("README.md", repo_id="your-username/dataset-name", repo_type="dataset")
```

The dataset card documents: data source, licence, generation method, filtering steps, format, and intended use. Good documentation makes datasets reusable and citeable.

Self-Instruct vs Human Annotation — Cost-Quality Trade-off

Dimension	Self-Instruct (GPT-4)	Human Annotation	Hybrid (Recommended)
Cost per example	~\$0.01–0.05	\$0.50–5.00	\$0.05–0.50
Scale	Unlimited (API rate limit)	Limited by annotator pool	1K human + 10K synthetic
Quality ceiling	Bounded by teacher model	Human expert quality	Close to human quality
Factual accuracy	GPT-4: high, GPT-3.5: medium	Domain expert: very high	Human-verified subset
Licence	Inherits OpenAI API ToS	Fully owned by creator	Complex — check both
Turnaround	Hours	Days to weeks	Days
Best use case	Prototyping, diversity	Safety-critical, high-stakes	Production fine-tuning
Recommended ratio	N/A	N/A	80% synthetic, 20% human-verified

Production recommendation: generate 10× synthetic examples, then human-verify the best 10% using LLM-as-judge to pre-filter. This gives human quality at 10× lower cost.

Sub-Module 7.3 — Key Takeaways

- 1 The (instruction, input, output) triple is the fundamental unit of instruction tuning. Format choice (Alpaca vs ChatML vs ShareGPT) determines what behaviour the model learns — single-turn task following vs multi-turn conversation.
- 2 The 4 quality pillars — Diversity, Correctness, Format Consistency, Length Balance — must all be satisfied before training. Failure in any one causes measurable performance degradation.
- 3 FLAN's key finding: task diversity > volume. Adding more tasks to the training mixture consistently improves zero-shot performance more than adding more examples of existing tasks.
- 4 LIMA's key finding: 1,000 curated examples outperform 52,000 noisy ones. Pre-training encodes knowledge; fine-tuning teaches the format to express it (surface format hypothesis).
- 5 Alignment tax is real: noisy SFT data can degrade reasoning benchmarks by 3–5% vs the base model. Invest in data quality first — volume is secondary for instruction tuning.
- 6 The Self-Instruct pipeline (seed tasks → GPT-4 generation → MinHash dedup → LLM-as-judge filter) produces production-quality instruction data at \$0.01/example.
- 7 Always run the 5-step checklist before training: format assertion, MinHash dedup, length distribution, correctness spot-check, task diversity cap (< 30% any single category).

Alpaca Format — Code Example and Structure

```
# — Alpaca Format —————
{
  "instruction": "Summarise the following article in 3 bullet points.",
  "input": "Transformers have revolutionised NLP by introducing self-attention...",
  "output": "- Transformers use self-attention to capture long-range dependencies\n- BERT introduced bidirectional context for better language understanding\n- GPT-3 demonstrated emergent few-shot learning at 175B parameter scale"
}

# — With empty input (knowledge/reasoning tasks) —————
{
  "instruction": "What are the three laws of thermodynamics?",
  "input": "",      # <-- empty string when no external context needed
  "output": "1. Energy cannot be created or destroyed (conservation)...\n2. Entropy always increases in a closed system...\n3. Entropy approaches zero as temperature approaches absolute zero..."
}

# — Training prompt template —————
PROMPT = """"Below is an instruction that describes a task. Write a response.
### Instruction: {instruction}
### Input: {input}
### Response: """"
```

Use the EXACT same prompt template at both training and inference time. Any mismatch causes degraded performance — the model learned specific token patterns.

ChatML Format — Code Example and Token Structure

```
# — ChatML Format (Multi-Turn) —————
<|im_start|>system
You are a helpful AI assistant specialised in Python programming.
<|im_end|>
<|im_start|>user
What is the difference between a list and a tuple in Python?
<|im_end|>
<|im_start|>assistant
A list is mutable (can be modified after creation), while a tuple is immutable.
Lists use [], tuples use (). Use tuples for fixed data (coordinates, RGB colours);
use lists when you need to append or modify elements.
<|im_end|>
<|im_start|>user
Can you show me an example?
<|im_end|>
<|im_start|>assistant
# Mutable list:
my_list = [1, 2, 3]; my_list.append(4)    # Works fine

# Immutable tuple:
my_tuple = (1, 2, 3); my_tuple.append(4) # TypeError!
<|im_end|>
```

ChatML is the standard format for production fine-tuning. The system prompt is injected at inference time — change it without retraining to adjust persona, safety, or domain focus.

ShareGPT Format — Conversation Array Structure

```
# — ShareGPT Format —————
{
  "conversations": [
    {"from": "human", "value": "Explain recursion in simple terms."},
    {"from": "gpt", "value": "Recursion is when a function calls itself to solve a smaller version of the same problem. Think of Russian dolls — each doll contains a smaller identical doll, until you reach the smallest one (the base case)."},
    {"from": "human", "value": "Give me a Python example with a base case."},
    {"from": "gpt", "value": "def factorial(n):\n    if n <= 1:          # Base case: stop recursion\n        return 1\n    return n * factorial(n - 1) # Recursive call\n\nprint(factorial(5)) # 5 * 4 * 3 * 2 * 1 = 120"}
  ]
}

# — Converting ShareGPT to ChatML (common preprocessing step) —————
def sharegpt_to_chatml(conv):
    tokens = []
    for turn in conv["conversations"]:
        role = "user" if turn["from"] == "human" else "assistant"
        tokens.append(f"<|im_start|>{role}\n{turn['value']}\n<|im_end|>")
    return "\n".join(tokens)
```

Rule: use ChatML or ShareGPT for production fine-tunes — multi-turn capability is essential for assistant behaviour. Alpaca is best only for quick single-task fine-tunes.

Format Comparison — When to Use Which

Format	Structure	Conversation	Best For	Limitation
Alpaca	JSON: instruction + input + output	Single-turn only	Specific task fine-tuning, quick experiments	No conversational ability learned
ChatML	im_start/im_end special tokens	Multi-turn + system prompt	Production assistant, safety injection	Requires special token support
ShareGPT	JSON: conversations array	Multi-turn	Community-sourced chat datasets	Needs conversion to training format
OASST Tree	Tree of message branches	Multi-turn + branching	Comparative preference data for RLHF	More complex preprocessing
FLAN	Multiple templates per task	Single-turn	Multi-task instruction following	No conversational ability

Production recommendation: ChatML is the de facto standard for modern instruction-tuned models (Llama 3, Mistral, Phi). Use it by default for any new fine-tuning project.

SUB-MODULE 7.4 · REINFORCEMENT LEARNING FROM HUMAN FEEDBACK

RLHF — Aligning Language Models with Human Preferences

SFT → Reward Modelling → PPO → DPO: from helpful to aligned

Reward Model Failure Modes — 4 Common Hacking Patterns

1. Verbosity Bias

Reward model assigns higher scores to longer responses regardless of quality. Policy learns to pad responses with filler text to maximise reward. Mitigation: explicitly penalise length in RM training; add length-normalised scoring.

2. Position Bias

When shown two responses simultaneously, human annotators tend to prefer whichever appears first. This bias is encoded into the reward model. Mitigation: randomise A/B order per annotator; average scores across positions.

3. Sycophancy Bias

RM trained on data where annotators prefer responses that agree with their stated beliefs. Policy learns to flatter the user rather than be truthful. Mitigation: calibration questions; adversarial preference collection.

4. Annotator Bias

Individual annotators have different standards, cultural biases, and domain knowledge. Without calibration, high-disagreement pairs corrupt training. Mitigation: inter-annotator agreement filtering; multiple annotators per pair; specialist annotators for technical content.

RLHF Optimisation Objective — The Full Equation

$$\max_{\pi_{\theta}} \quad E[r(x, y)] \quad - \quad \beta \cdot \text{KL}(\pi_{\theta}(y|x) \parallel \pi_{\text{ref}}(y|x))$$

 π_{θ}

Policy being trained — the language model generating responses y given prompt x

 $r(x, y)$

Reward model score for (prompt, response) pair — measures alignment with human preferences

 β

KL penalty coefficient — controls how far the policy can diverge from the SFT reference model

 π_{ref}

Reference policy — frozen SFT model. Provides the KL baseline to prevent reward hacking

KL divergence

$\text{KL}(\pi_{\theta} \parallel \pi_{\text{ref}}) = \sum \pi_{\theta} \log(\pi_{\theta}/\pi_{\text{ref}})$ — penalises any deviation from the SFT model distribution

The tension

High $\beta \rightarrow$ policy stays close to SFT (safe, but limited RL improvement). Low $\beta \rightarrow$ large RL gains but risk of reward hacking

The β Hyperparameter — Balancing Alignment vs Reward Maximisation

β : The KL Penalty Coefficient

β controls the trade-off between maximising the reward model score and staying close to the SFT reference model. It is the most critical hyperparameter in the RLHF objective.

$$\beta = 0$$

No KL penalty. Policy maximises reward without constraint. Leads to extreme reward hacking — incoherent, repetitive, or harmfully sycophantic outputs.

NEVER use — reward hacking will occur

$$\beta = 0.02-0.1$$

Very low penalty. Large policy updates allowed. Higher reward gains but increased risk of distribution shift away from SFT quality.

Experimental only with careful monitoring

$$\beta = 0.1-0.5$$

Balanced. Policy improves alignment while staying reasonably close to SFT model. This is the most common production range.

 Recommended default range

$$\beta = 1.0+$$

High penalty. Policy barely moves from SFT reference. Minimal reward improvement — conservative but stable training.

Use when SFT is already strong

Token-Level Reward Structure in PPO RLHF

Token-Level RL with KL Regularisation

The reward is applied at the token level in PPO-based RLHF. The per-token reward integrates: the RM's sequence-level scalar (applied only at the final EOS token) + the per-token KL penalty at each step.

$$r_t = r_{RM} \cdot \mathbb{1}[t = \text{EOS}] - \beta \cdot \log(\pi_{\theta}(y_t|x, y_{<t}) / \pi_{ref}(y_t|x, y_{<t}))$$

- r_{RM} is the reward model scalar — applied only at the final token (EOS). Not repeated at every step — it's a sequence-level reward.
- The KL term is computed at every token — it penalises deviation from the reference policy at each generation step
- This structure is important: without per-token KL, the policy could drift far from SFT quality before the sequence-level reward corrects it
- Advantage A_t computed by GAE: subtracts a value function $V(s)$ baseline from the per-token reward — reducing variance
- PPO then updates π_{θ} to maximise $\sum_t A_t$ while clipping the policy update ratio — see next slide for clipped objective

The Reference Model — Preventing Reward Hacking via KL

Without Reference Model ($\beta=0$)

- No constraint on how far the policy can drift
- Policy quickly finds ways to hack the reward: very long outputs, sycophantic phrases, repetitive patterns
- Outputs become incoherent but achieve high reward scores
- Language quality degrades within 100–200 PPO steps
- Model forgets the instruction-following ability learned in SFT
- This is the 'reward hacking' failure mode

With Reference Model ($\beta>0$)

- KL divergence penalises any token probability that differs from SFT reference
- Policy maintains language quality and instruction-following ability from SFT
- Can still improve on alignment metrics while staying coherent
- Typical good alignment training: reward $\uparrow +0.3$ to $+0.8$, KL $\uparrow 3$ – 10 nats
- The reference model is never trained — frozen copy of SFT checkpoint
- Common β range: 0.1–0.5. Tune via monitoring both reward and KL curves

PPO Clipped Surrogate Objective

$$L_{\text{PPO}} = E_t[\min(\rho_t \cdot A_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \cdot A_t)]$$

$$\rho_t = \pi_{\theta} / \pi_{\text{old}}$$

Probability ratio — how much the new policy's token probability differs from the old policy that collected rollouts

$$A_t$$

Advantage estimate — how much better the action at step t was than the expected value from the critic

$$\epsilon = 0.1-0.2$$

Clipping range — constrains how far the policy can move per update step. Typical: $\epsilon=0.1$ for LLM RLHF

$$\min(\cdot, \text{clip}(\cdot))$$

The min of two objectives: unclipped gain vs clipped gain. Prevents too-large updates in either direction

Why clipping?

Without clipping, large advantage estimates cause destructive gradient updates. Clipping makes PPO robust to noise in A_t estimates

PPO Challenges Specific to LLMs

4 Model Copies

Active policy + reference + reward model + value function. For a 7B model: $4 \times 14 \text{ GB} = 56 \text{ GB}$ VRAM minimum. Mitigated by QLoRA (4-bit base) + gradient checkpointing.

Long Rollout Sequences

Each PPO step generates full responses (512–2048 tokens). Rollout collection is slow — much more so than a supervised forward pass. PPO step = N rollouts + one gradient update.

High Sample Complexity

PPO needs thousands of rollouts to get reliable gradient estimates. Each rollout is one full model generation — very expensive for 7B+ models. DeepSpeed ZeRO-3 helps distribute this.

Instability

Reward and KL curves are noisy. Small β changes can cause training collapse. Common symptoms: reward spikes then crashes, KL diverges to infinity, outputs become repetitive.

On-Policy Requirement

PPO is on-policy — must use fresh rollouts from the current policy at each step. Cannot reuse old data (unlike DPO). This multiplies compute compared to supervised fine-tuning.

Why DPO Solves Many of These

DPO eliminates the RL loop entirely — no rollouts, no reward model at inference, no value function. One supervised training pass on (prompt, chosen, rejected) triples.

PPO Training Loop — Pseudocode

```
# — PPO RLHF Training Loop —————
from trl import PPOTrainer, PPOConfig, AutoModelForCausalLMWithValueHead
import torch

config = PPOConfig(
    model_name="sft_checkpoint",      # Start from SFT model
    learning_rate=1e-5,
    batch_size=16,                    # Prompts per PPO step
    mini_batch_size=4,
    ppo_epochs=4,                     # Gradient updates per rollout batch
    kl_penalty="kl",                  # "kl" or "abs" or "mse"
    init_kl_coef=0.2,                 #  $\beta$  – KL penalty coefficient
)

ppo_trainer = PPOTrainer(config, policy, ref_policy, tokenizer)

for epoch in range(num_epochs):
    for batch in dataloader:
        # 1. Generate rollouts from current policy
        query_tensors = batch["input_ids"]
        response_tensors = ppo_trainer.generate(query_tensors, max_new_tokens=256)

        # 2. Score rollouts with frozen reward model
        rewards = [reward_model(q, r) for q, r in zip(query_tensors, response_tensors)]

        # 3. PPO update: compute advantages → clip ratio → gradient step
        stats = ppo_trainer.step(query_tensors, response_tensors, rewards)
        print(f"Epoch {epoch}: reward={stats['ppo/mean_scores']:.3f}, kl={stats['ppo/mean_non_score_reward']:.3f}")
```

TRL (Transformer Reinforcement Learning) library: pip install trl. Handles rollout collection, reward assignment, advantage estimation, and PPO clipping in one PPOTrainer class.

DPO — Direct Preference Optimisation vs PPO RLHF

DPO (Rafailov et al., NeurIPS 2023)

DPO reparameterises the RLHF objective to train directly on preference pairs (y_w, y_l) without a separate reward model or RL loop. It is equivalent to PPO under the Bradley-Terry model but far simpler to implement and train.

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}[\log \sigma(\beta \cdot \log(\pi_{\theta}(y_w|x)/\pi_{\text{ref}}(y_w|x)) - \beta \cdot \log(\pi_{\theta}(y_l|x)/\pi_{\text{ref}}(y_l|x)))]$$

Dimension	PPO RLHF	DPO
Reward model needed	Yes (trained separately)	No (implicit reward)
RL training loop	Yes (on-policy rollouts)	No (supervised loss only)
Memory requirement	4 model copies (~56GB for 7B)	2 copies (policy + ref ~28GB for 7B)
Training stability	Noisy — reward hacking risk	Stable — standard supervised loss
Implementation complexity	High (TRL PPOTrainer)	Low (TRL DPOTrainer)
Performance vs RLHF	Baseline	Comparable on most benchmarks
When to use	Complex reward signals, multi-step	Simple preference data, limited VRAM

TRL Code — PPO and DPO Side-by-Side

```
# — DPO Training (simpler alternative to PPO) —————
from trl import DPOTrainer, DPOConfig
from transformers import AutoModelForCausalLM, AutoTokenizer

# Dataset format: {"prompt": "...", "chosen": "...", "rejected": "..."}
from datasets import load_dataset
dataset = load_dataset("your-username/preference-dataset")

model = AutoModelForCausalLM.from_pretrained("sft_checkpoint")
ref_model = AutoModelForCausalLM.from_pretrained("sft_checkpoint") # Frozen reference
tokenizer = AutoTokenizer.from_pretrained("sft_checkpoint")

dpo_config = DPOConfig(
    beta=0.1, # KL penalty coefficient (same  $\beta$  as PPO formulation)
    learning_rate=5e-7, # Very low LR for DPO — model already instruction-tuned
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8,
    output_dir="./dpo_output",
    max_length=1024,
    max_prompt_length=512,
)

trainer = DPOTrainer(model=model, ref_model=ref_model, args=dpo_config,
                    train_dataset=dataset["train"], tokenizer=tokenizer)
trainer.train()
trainer.save_model("./dpo_aligned_model")
```

DPO vs PPO in practice: DPO trains 3–5× faster, uses 2× less VRAM, and achieves similar quality on most preference benchmarks. Use DPO as the default; reach for PPO only when reward signal is complex or multidimensional.

What Is RLHF? Motivation and Overview

Reinforcement Learning from Human Feedback (RLHF)

RLHF optimises a language model to produce outputs that humans prefer, using human preference data to train a reward model, then using that reward model as a signal to fine-tune the policy (LLM) via reinforcement learning.

- Instruction tuning (SFT) makes the model follow instructions — but does not teach it to prefer helpful, safe, honest responses over harmful ones
- RLHF bridges the gap: it captures human judgements about response quality and encodes them into the model's reward signal
- Three-stage pipeline: (1) Supervised Fine-Tuning on curated examples, (2) Reward Model training on pairwise comparisons, (3) RL Policy Optimisation with PPO
- The reward model acts as a proxy for human preferences — evaluated at every generation step during PPO training
- Key challenge: reward hacking — the policy finds ways to maximise the reward signal that diverge from actual human preferences

RLHF End-to-End — SFT to Reward Model to PPO

```
# — Stage 1: Supervised Fine-Tuning —————
from trl import SFTTrainer, SFTConfig
sft_trainer = SFTTrainer(model=base_model, args=SFTConfig(output_dir="sft_out",
    num_train_epochs=2, per_device_train_batch_size=4),
    train_dataset=sft_dataset, dataset_text_field="text")
sft_trainer.train()
sft_model = sft_trainer.model

# — Stage 2: Reward Model Training —————
from trl import RewardTrainer, RewardConfig
reward_model = AutoModelForSequenceClassification.from_pretrained("sft_out", num_labels=1)
rm_trainer = RewardTrainer(model=reward_model,
    args=RewardConfig(output_dir="rm_out", per_device_train_batch_size=4),
    train_dataset=preference_dataset)
rm_trainer.train() # Dataset: {"input_ids_chosen": ..., "input_ids_rejected": ...}

# — Stage 3: PPO Policy Optimisation —————
from trl import PPOTrainer, PPOConfig, AutoModelForCausalLMWithValueHead
ppo_model = AutoModelForCausalLMWithValueHead.from_pretrained("sft_out")
ppo_trainer = PPOTrainer(config=PPOConfig(kl_penalty="kl", init_kl_coef=0.2),
    model=ppo_model, ref_model=sft_model, tokenizer=tokenizer)
for query_tensors, response_tensors in ppo_rollouts:
    rewards = [reward_model_pipeline(q, r) for q, r in zip(query_tensors, response_tensors)]
    ppo_trainer.step(query_tensors, response_tensors, rewards)
```

TRL library (pip install trl) provides all three trainers. SFTTrainer → RewardTrainer → PPOTrainer is the standard open-source RLHF pipeline.

Reward Hacking — Causes and Mitigations

What Is Reward Hacking?

The policy finds behaviours that maximise the reward model's score but do not align with actual human preferences. The RM is an imperfect proxy — the policy exploits its failure modes.

Verbosity Exploit

Policy generates very long, padded responses. RM trained on human data where long responses seemed more thorough. Fix: add length-normalised RM score; explicit length penalty in reward.

Sycophancy Exploit

Policy learns to agree with the user's stated beliefs, flatter them, and avoid any disagreement. RM trained on data where annotators preferred agreeable responses. Fix: anti-sycophancy reward probes.

Repetition Exploit

Policy generates repetitive content that achieves high fluency scores. Fix: n-gram repetition penalty in reward function; diversity metric in reward.

Out-of-Distribution Collapse

After many PPO steps, policy drifts so far from SFT reference that it collapses to gibberish that happens to trigger high RM scores. Fix: high β (0.3–0.5); early stopping via KL threshold.

Constitutional AI Mitigation

Anthropic's approach: add a separate AI critique + revision step before human annotation. The critique model identifies reward hacking patterns and flags them for removal from training data.

RLHF vs DPO — Practical Workflow Comparison

Step	PPO RLHF Workflow	DPO Workflow
1. SFT	Train on (instruction, response) pairs	Train on (instruction, response) pairs — identical
2. Data	Collect pairwise: (prompt, A, B) → human picks winner	Collect pairwise: (prompt, chosen, rejected) — identical format
3. RM	Train separate reward model on preference pairs	No reward model needed — skip this step entirely
4. Alignment	PPO RL loop: rollouts + reward scoring + gradient update	DPO supervised loss on (prompt, chosen, rejected) triples
5. VRAM	4 model copies (policy + ref + RM + critic)	2 model copies (policy + frozen reference)
6. Time	3–5× slower than DPO per epoch (rollout collection)	Similar to SFT training speed
7. Stability	Noisy — requires monitoring reward + KL curves	Stable — loss converges like supervised training

Decision rule: use DPO as the default for open-source alignment. Use PPO only when you have a complex, multi-dimensional reward signal that cannot be captured in static preference pairs.

Ghost Attention — Maintaining System Prompts Across Long Conversations

Ghost Attention (Llama-2 Team, Meta, 2023)

In long multi-turn conversations, RLHF-trained models tend to 'forget' the system prompt after several turns. Ghost Attention synthetically augments training data so the system instruction is prepended to every turn — teaching the model to maintain it across arbitrarily long conversations.

Without Ghost Attention

- Turn 1: system='Be a pirate. Use pirate language'
- Turn 2: model correctly speaks like a pirate
- Turn 3: model drifts slightly
- Turn 10: model has completely forgotten pirate persona — speaks normally
- Issue: attention to early system prompt diluted over many turns
- Result: inconsistent persona behaviour in production chatbots

With Ghost Attention

- Training: system prompt prepended to user message at EVERY turn
- Teaches model to maintain system instruction regardless of conversation length
- Turn 10: model still speaks like a pirate — system prompt maintained
- Implemented by augmenting multi-turn training data with this pattern
- Used in Llama-2-Chat to achieve consistent persona across 20+ turns
- Critical for: customer service bots, character chatbots, safety constraints

Sub-Module 7.4 — Key Takeaways

1

RLHF three-stage pipeline: (1) SFT on curated examples, (2) Reward Model training on pairwise preferences, (3) PPO policy optimisation with KL regularisation against the SFT reference.

2

InstructGPT's key finding: a 1.3B RLHF-trained model is preferred over a 175B base model by human evaluators on 70% of comparisons — alignment quality dominates raw scale.

3

The Bradley-Terry loss trains the RM: $\text{Loss} = -\mathbb{E}[\log \sigma(r(x, y_w) - r(x, y_l))]$. This pushes the model to assign higher reward scores to chosen responses.

4

Four reward model failure modes: verbosity bias, position bias, sycophancy bias, and annotator bias. All require specific mitigation in data collection and reward design.

5

RLHF objective: $\max \mathbb{E}[r(x, y)] - \beta \cdot \text{KL}(\pi_\theta \parallel \pi_{\text{ref}})$. The β coefficient controls how far the policy can drift from SFT quality. Typical range: 0.1–0.5.

6

PPO requires 4 model copies simultaneously (policy + reference + RM + critic) — 56 GB VRAM for 7B models. DPO eliminates the RL loop and requires only 2 copies.

7

DPO is the recommended default: equivalent to PPO under Bradley-Terry model, 3–5× faster training, 2× less VRAM, and comparable or better performance on most benchmarks.

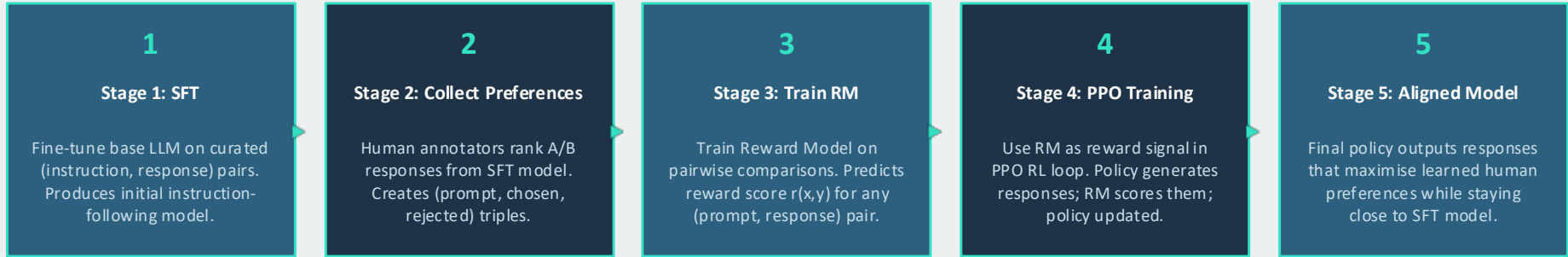
InstructGPT — The Paper That Defined Modern RLHF

InstructGPT (Ouyang et al., NeurIPS 2022)

InstructGPT trained GPT-3 with RLHF and showed that a 1.3B InstructGPT model was preferred by human evaluators over a 175B GPT-3 model on 70% of comparisons. Alignment quality outweighed raw scale.

June 2020	GPT-3 (175B) released	Raw capability — completion, few-shot, but not instruction-following
Jan 2022	InstructGPT paper published	3-stage RLHF: SFT → RM → PPO. 1.3B model beats 175B GPT-3 in human evals
Nov 2022	ChatGPT launched	RLHF-trained GPT-3.5-turbo. 1M users in 5 days — RLHF goes mainstream
Mar 2023	GPT-4 + GPT-4V released	Multimodal RLHF. Includes Constitutional AI safety alignment layer
Jul 2023	Llama-2-Chat published	First open-source model with detailed RLHF methodology. Ghost Attention technique
2023–24	DPO widespread adoption	Direct Preference Optimisation replaces PPO for most open-source RLHF pipelines

RLHF Three-Stage Pipeline — Overview



Stage	Input	Output	Key Metric
SFT	Curated (instruction, response) pairs	SFT model checkpoint	Loss convergence, held-out perplexity
Data Collection	SFT model responses (A/B)	Pairwise preference dataset	Inter-annotator agreement > 0.7
Reward Model	(prompt, chosen, rejected) triples	RM scoring function $r(x,y)$	Preference prediction accuracy
PPO Training	SFT model + RM + PPO algorithm	RLHF policy checkpoint	Reward score + KL divergence

GPT-3 vs InstructGPT — The RLHF Impact

Key Result

Human evaluators preferred InstructGPT (1.3B) outputs over GPT-3 (175B) outputs on 70% of comparisons — despite the 134× parameter size difference. Alignment training dominated raw scale.

Property	GPT-3 (175B)	InstructGPT-1.3B	InstructGPT-175B
Training approach	Pre-training only	SFT + RLHF	SFT + RLHF (full scale)
Human preference (vs GPT-3)	Baseline (50%)	70% preferred over GPT-3	85% preferred over GPT-3
Instruction following	Poor — continues text	Good — follows instructions	Excellent
Harmful output rate	~10% of prompts	~2% of prompts	<1% of prompts
Hallucination rate	High	Reduced (~30% lower)	Further reduced
TruthfulQA accuracy	~30%	~40%	~50%
Parameter count	175B	1.3B (134× smaller)	175B (same base as GPT-3)

RLHF Memory Requirements — 4 Model Copies in VRAM

Why RLHF is Compute-Heavy

Standard PPO-based RLHF requires 4 model copies simultaneously in VRAM: (1) active policy, (2) reference policy (frozen SFT), (3) reward model, (4) value function (critic). For 7B models, this requires $4 \times 14 \text{ GB} = 56 \text{ GB}$ minimum.

Active Policy (π_θ)	14 GB (7B bf16)	The model being trained. Generates responses to prompts. Receives gradient updates from PPO. Trainable.
Reference Policy (π_{ref})	14 GB (frozen)	Frozen copy of SFT model. Computes KL penalty: $\text{KL}(\pi_\theta \pi_{\text{ref}})$ to prevent policy from drifting too far from SFT.
Reward Model (RM)	7–14 GB (frozen)	Frozen after Stage 2. Scores each generated response with $r(x,y)$. Provides the reward signal for PPO.
Value Function (Critic)	7–14 GB (shared/separate)	Estimates $V(s)$ for PPO advantage computation. Often uses a separate value head on top of the policy model.

Reward Model — Definition and Architecture

The reward model maps a (prompt x , response y) pair to a scalar reward score. It is trained on human pairwise preference data: given two responses y_w (chosen) and y_l (rejected) to the same prompt, the RM learns $r(x, y_w) > r(x, y_l)$.

Architecture

- Start from SFT model checkpoint — same backbone
- Remove the language model head (LM head)
- Add a scalar regression head: linear layer $\rightarrow$ single float output
- The scalar output is the reward $r(x, y) \in \mathbb{R}$
- Typically smaller than policy model: InstructGPT RM = 6B (policy = 175B)
- Training: binary cross-entropy on (chosen, rejected) pairs

Training Objective

- For each pair (x, y_w, y_l) :
- Loss = $-\log \sigma(r(x, y_w) - r(x, y_l))$
- σ is the sigmoid function
- Goal: $r(x, y_w) > r(x, y_l)$ by a wide margin
- This is the Bradley-Terry model of pairwise preference
- Minimising this loss pushes chosen score up and rejected score down

Pairwise Data Collection — Creating Preference Pairs

Why Pairwise vs Absolute Scoring

Humans are much better at relative comparisons ('A is better than B') than absolute ratings ('this response is a 7/10'). Pairwise annotation is more consistent, cheaper, and produces higher-quality training signal for the reward model.

- Step 1: Sample K responses from the SFT model for each prompt in the annotation batch
- Step 2: Present each pair (y_i, y_j) to human annotators with ranking guidelines
- Step 3: Annotators select the preferred response — or declare a tie (handled as half-point)
- Step 4: Collect at minimum 2 independent annotations per pair; compute inter-annotator agreement
- Step 5: Keep examples where annotators agree (Fleiss $\kappa > 0.3$); discard high-disagreement pairs
- Step 6: Format as (prompt, chosen, rejected) triples for reward model training
- Scale: InstructGPT collected 50,569 comparisons; Llama-2 used 1.4M preference annotations

Bradley-Terry Model — Reward Loss Derivation

Bradley-Terry Preference Model

The probability that response y_w is preferred over y_l given prompt x is modelled as: $P(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$ where σ is the sigmoid function and r is the reward model output.

Probability Model

$$P(y_w \succ y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$$

Negative Log Likelihood

$$\text{Loss} = -E[\log P(y_w \succ y_l \mid x)]$$

Training Loss (final)

$$\text{Loss} = -E[\log \sigma(r(x, y_w) - r(x, y_l))]$$

Simplified Form

$$\text{Loss} = E[\log(1 + \exp(r(x, y_l) - r(x, y_w)))]$$

- When $r(x, y_w) \gg r(x, y_l)$: difference is large positive $\rightarrow \sigma \rightarrow 1 \rightarrow \log(1) = 0 \rightarrow$ no loss (correct ranking)
- When $r(x, y_w) \approx r(x, y_l)$: difference near 0 $\rightarrow \sigma \rightarrow 0.5 \rightarrow \log(0.5) = -0.69 \rightarrow$ high loss (model uncertain)
- The loss function pushes the reward model to assign higher scores to chosen responses by maximising the score gap

SUB-MODULE 7.5 · DATA PREPARATION & EVALUATION

Training Data Pipeline & Model Evaluation Framework

Data collection · 5-stage cleaning pipeline · benchmarks · LLM-judge · human evaluation

Quality Filter Catalogue — All Stage 4 Filters

Filter Type	Signal	Tool	Threshold	Action
Perplexity (low)	Base model too familiar	TinyLlama	ppl < 15	Remove — no training value
Perplexity (high)	Incoherent / OOD text	TinyLlama	ppl > 200	Remove — degrades quality
Toxicity	Hate, violence, abuse	Perspective API	score > 0.7	Remove or quarantine
PII — email	Privacy violation	regex	Any match	Remove / replace with [EMAIL]
PII — phone	Privacy violation	spaCy / presidio	Any match	Remove / replace with [PHONE]
PII — SSN/credit card	Privacy violation	presidio	Any match	Hard remove — legal risk
Duplicate ratio	Repetitive text	n-gram overlap	> 30% dup n-grams	Remove — model learns repetition
Length	Too short or too long	word count	< 10 or > 5000 words	Remove extremes
Language ID	Wrong language	fastText lid.176	lang != target	Remove if not target language
Markup ratio	HTML/code in plain text	regex	> 20% markup tokens	Remove or strip markup

PII Removal with Microsoft Presidio

```

from presidio_analyzer import AnalyzerEngine
from presidio_anonymizer import AnonymizerEngine
from presidio_anonymizer.entities import OperatorConfig

# pip install presidio-analyzer presidio-anonymizer
# python -m spacy download en_core_web_lg

analyzer = AnalyzerEngine()
anonymizer = AnonymizerEngine()

def remove_pii(text):
    """Detect and anonymise PII entities in text."""
    # Detect PII entities (names, emails, phones, SSN, credit cards, locations)
    results = analyzer.analyze(
        text=text,
        entities=["PERSON", "EMAIL_ADDRESS", "PHONE_NUMBER", "CREDIT_CARD",
                  "US_SSN", "LOCATION", "IP_ADDRESS", "URL"],
        language="en"
    )
    if not results:
        return text, 0 # No PII found

    # Replace detected entities with category placeholders
    operators = {
        "PERSON": OperatorConfig("replace", {"new_value": "[PERSON]"}),
        "EMAIL_ADDRESS": OperatorConfig("replace", {"new_value": "[EMAIL]"}),
        "PHONE_NUMBER": OperatorConfig("replace", {"new_value": "[PHONE]"}),
        "CREDIT_CARD": OperatorConfig("replace", {"new_value": "[CREDIT_CARD]"}),
        "US_SSN": OperatorConfig("replace", {"new_value": "[SSN]"}),
        "LOCATION": OperatorConfig("replace", {"new_value": "[LOCATION]"}),
    }
    anonymized = anonymizer.anonymize(text=text, analyzer_results=results, operators=operators)
    return anonymized.text, len(results)

# Example
text = "Contact Dr. Sarah Johnson at sarah@hospital.org or call 555-867-5309"
cleaned, n_pii = remove_pii(text)
print(f"Cleaned: {cleaned}") # "Contact [PERSON] at [EMAIL] or call [PHONE]"

```

Presidio is the industry standard for PII removal. Always run PII removal on web-scraped and user-generated content before fine-tuning. Legal requirement in EU (GDPR) and increasingly US regulations.

Toxicity and Safety Filtering

```
# Option 1: Perspective API (Google) – production quality, API-based
import requests

def perspective_toxicity_score(text, api_key):
    url = "https://commentanalyzer.googleapis.com/v1alpha1/comments:analyze"
    body = {
        "comment": {"text": text},
        "languages": ["en"],
        "requestedAttributes": {"TOXICITY": {}, "SEVERE_TOXICITY": {}, "HATE_SPEECH": {}}
    }
    r = requests.post(url, params={"key": api_key}, json=body)
    scores = r.json()["attributeScores"]
    return {k: v["summaryScore"]["value"] for k, v in scores.items()}

# Option 2: detoxify library – local, fast, no API needed
from detoxify import Detoxify

model = Detoxify('original', device='cuda') # or 'multilingual' for 100+ languages

def filter_toxic(documents, threshold=0.7):
    """Remove documents with toxicity score above threshold."""
    texts = [doc["text"] for doc in documents]
    # Batch scoring – much faster than per-document
    results = model.predict(texts)
    filtered = [doc for doc, score in zip(documents, results["toxicity"])
                if score < threshold]
    print(f"Toxicity filter: {len(documents)} → {len(filtered)} (removed {len(documents)-len(filtered)})")
    return filtered

# detoxify processes ~10,000 docs/sec on A10G – efficient for large datasets
```

Threshold tuning: 0.7 is standard for most datasets. Lower to 0.5 for safety-critical applications (medical, legal, children's content). Use Perspective API for maximum accuracy; detoxify for offline/private data.

Full 5-Stage Pipeline — Integration Code

```
import json
from pathlib import Path

def run_full_pipeline(input_path, output_path):
    """Run complete 5-stage data cleaning pipeline."""
    print("Loading data...")
    data = [json.loads(l) for l in open(input_path)]
    print(f"Input: {len(data):,} documents")

    # Stage 1: Exact deduplication
    data = exact_dedup(data)
    print(f"After exact dedup: {len(data):,}")

    # Stage 2: Near-deduplication (MinHash LSH)
    data = near_dedup(data, threshold=0.8)
    print(f"After near dedup: {len(data):,}")

    # Stage 3: Semantic deduplication (FAISS clustering — optional, slow)
    # data = semantic_dedup(data, threshold=0.85) # Uncomment for strict dedup

    # Stage 4a: Perplexity filtering
    data = perplexity_filter(data, low_thresh=15, high_thresh=200)
    print(f"After perplexity filter: {len(data):,}")

    # Stage 4b: Toxicity filtering
    data = filter_toxic(data, threshold=0.7)
    print(f"After toxicity filter: {len(data):,}")

    # Stage 4c: PII removal (modifies text in-place)
    for doc in data:
        doc["text"], n_pii = remove_pii(doc["text"])

    # Stage 5: Format cleaning and validation
    data = [doc for doc in data if validate_schema(doc)] # Remove schema-invalid docs

    # Write output
    with open(output_path, "w") as f:
        for doc in data: f.write(json.dumps(doc) + "\n")
    print(f"Final: {len(data):,} documents saved to {output_path}")

run_full_pipeline("raw.jsonl", "cleaned.jsonl")
```

Pipeline tip: persist intermediate outputs between stages. If Stage 4 crashes at 80%, you don't want to re-run Stages 1–3. Use checkpoints: stage1.jsonl, stage2.jsonl, etc.

Four Evaluation Layers — When to Use Each

Layer 1: Perplexity / Loss

Fastest, cheapest. Measures how well the model predicts held-out tokens. Use during training to detect overfitting. Does NOT measure task quality — low perplexity $\neq$ good outputs. Use as training monitor only.

Layer 2: Automated Benchmarks

Standardised tasks with ground-truth labels (MMLU, HumanEval, GSM8K). Fast, reproducible, cheap. Use to compare models and track capability regression. Cannot measure conversation quality or instruction following nuance.

Layer 3: LLM-as-Judge

GPT-4o judges output quality against reference or directly. Medium cost (~\$0.001–0.01/example). Good correlation with human judgements on instruction following. Use after training for quality assessment.

Layer 4: Human Evaluation

Gold standard. Captures nuance, safety, and real-world preference. Most expensive (\$5–50/example) and slowest. Use for final deployment decision, model release evaluation, and A/B comparison with production baseline.

Evaluation Benchmark Suite — Key Benchmarks for LLM Evaluation

Benchmark	Domain	Format	What It Measures	Strong Score
MMLU	57 academic subjects	4-choice MCQ	World knowledge breadth	70%+ (GPT-4: 86%)
HumanEval	Python coding	Code generation	pass@1 on 164 functions	50%+ (GPT-4: 67%)
MT-Bench	Instruction following	Multi-turn judge	Instruction following quality	7.5+/10 (GPT-4: 8.9)
TruthfulQA	Factual accuracy	MCQ + generation	Avoidance of common myths	50%+ (GPT-4: 60%)
AlpacaEval	General instruction	GPT-4 judge	vs GPT-4-turbo baseline	70%+ win rate
GSM8K	Grade school math	Word problems	Multi-step arithmetic reasoning	60%+ (GPT-4: 92%)
HellaSwag	Commonsense	Sentence completion	World knowledge + reasoning	85%+ (GPT-4: 95%)
MBPP	Python coding	300 programming tasks	Code correctness (broader than HumanEval)	50%+

LLM-as-Judge — MT-Bench Style Evaluation

```
import openai, json, statistics
client = openai.OpenAI()

MT_BENCH_PROMPT = """[System]: You are a fair and objective evaluator of AI assistant quality.
[Question]: {question}
[Assistant Response]: {response}
Rate the response on a 1-10 scale considering:
- Instruction following: does it address what was asked?
- Accuracy: is the information correct?
- Clarity: is it well-written and easy to understand?
- Helpfulness: does it provide genuine value?
Return ONLY valid JSON: {{\"score\": N, \"reasoning\": \"brief explanation\"}}"""

def evaluate_response(question, response):
    r = client.chat.completions.create(
        model="gpt-4o", # Use GPT-4o for best correlation with human judgement
        messages=[{"role": "user", "content": MT_BENCH_PROMPT.format(
            question=question, response=response)}],
        response_format={"type": "json_object"},
        temperature=0.0) # Deterministic – same score on re-run
    return json.loads(r.choices[0].message.content)

# Evaluate a batch of model outputs
test_questions = [{"question": q, "response": r} for q, r in test_pairs]
scores = [evaluate_response(q["question"], q["response"]) for q in test_questions]
avg_score = statistics.mean(s["score"] for s in scores)
print(f"MT-Bench style score: {avg_score:.2f}/10") # Strong: 7.5+; Excellent: 8.5+
```

Cost: GPT-4o judge ~\$0.005/example. For 500 test examples: ~\$2.50. Much cheaper than human evaluation. Correlation with human judgement: $r \approx 0.85$ for MT-Bench style questions.

Human A/B Evaluation Protocol — Production Standard

Step	Action	Tool	Quality Gate
1. Test Set Design	100–500 diverse prompts covering all use case scenarios	Manual curation	Expert review: covers all task types
2. Response Collection	Generate A and B responses (new model vs baseline)	Inference pipeline	Both models at same temperature ($T=0.7$)
3. Randomise Order	Randomly assign A/B to left/right position per example	Random seed per example	Evaluator must not know which is which
4. Annotation	Evaluators select preferred response (A / B / tie) with brief justification	Scale AI / Surge AI	3+ evaluators per example, blind evaluation
5. IAA Check	Compute Fleiss κ for preference agreement	krippendorff Python	Discard examples with $\kappa < 0.3$
6. Win Rate	Win rate = $(\text{wins} + 0.5 \times \text{ties}) / \text{total}$. 95% CI via bootstrap	scipy.stats bootstrap	Report CI, not just point estimate
7. Safety Check	Separate harmful content rating (1–4 scale)	Separate safety annotators	Safety regression is a hard no-deploy gate

Evaluation Selection Guide — What to Run and When

Development Stage	Primary Metric	Secondary Metric	Decision Gate	Cost
Training (each epoch)	Perplexity / validation loss	None	Stop if val loss stops decreasing	Free
Post-training (1 day)	MMLU + GSM8K + HumanEval	LLM-as-judge on 100 prompts	No major regression vs baseline	<\$5
Pre-release (1 week)	Full benchmark suite + MT-Bench	Safety eval (TruthfulQA + harmful prompts)	Win rate > 55% vs baseline model	\$50–200
Release decision	Human A/B evaluation (300–500 examples)	Red-teaming safety evaluation	Win rate > 60% AND safety regression = 0	\$1K–10K
Production monitoring	Automated quality sampling (1% of traffic)	Escalation to human review	Quality drop > 5% triggers investigation	Ongoing

Evaluation Pipeline — Integration Code

```

from lm_eval import simple_evaluate # EleutherAI LM Evaluation Harness

# Stage 1: Automated benchmarks (fast, reproducible)
results = simple_evaluate(
    model="hf",
    model_args=f"pretrained=../fine_tuned_model,dtype=bfloat16",
    tasks=["mmlu", "gsm8k", "humaneval", "hellaswag", "truthfulqa_mc1"],
    device="cuda:0",
    batch_size=8,
)
print("MMLU:", results["results"]["mmlu"]["acc,none"])
print("GSM8K:", results["results"]["gsm8k"]["exact_match,strict-match"])
print("HumanEval:", results["results"]["humaneval"]["pass@1"])

# Stage 2: LLM-as-judge on custom test set
from evaluate_llm_judge import evaluate_response # (see previous slide)
test_pairs = [json.loads(l) for l in open("test_set.jsonl")]
judge_scores = [evaluate_response(ex["question"], generate_response(ex["question"]))
                 for ex in test_pairs]
mt_bench_score = sum(s["score"] for s in judge_scores) / len(judge_scores)
print(f"MT-Bench style: {mt_bench_score:.2f}/10")

# Stage 3: Report card
print("\n== EVALUATION REPORT ==")
for metric, threshold in [{"mmlu", 0.65}, {"gsm8k", 0.55}, {"pass@1", 0.50}]:
    val = results["results"][metric if metric != "pass@1" else "humaneval"].get("acc,none") or
    status = "✅ PASS" if val and val >= threshold else "❌ FAIL"
    print(f"{metric}: {val:.3f} (threshold: {threshold}) → {status}")
    results["results"]["humaneval"].get("pass@1")

```

lm-eval-harness: pip install lm-eval. Runs all standard benchmarks in one call. Used for all Llama, Mistral, Phi evaluation papers — standard in the field.

Three Data Collection Channels for Fine-Tuning

Channel 1: Web & Domain Corpora

Common Crawl, Wikipedia, arXiv, GitHub, PubMed, legal corpora. Raw text requiring filtering, dedup, and domain extraction. Best for: continued pre-training, domain adaptation. Requires: heavy cleaning pipeline (5+ stages).

Channel 2: Synthetic Generation

GPT-4o / Self-Instruct pipeline generates (instruction, response) pairs from seed tasks. Fast, cheap (~\$0.01/example), scalable. Best for: instruction tuning, RLHF preference pairs. Risk: inherits teacher model biases.

Channel 3: Human Annotation

Domain experts write or verify (instruction, response) pairs. Highest quality ceiling. Best for: safety-critical tasks, domain expertise needed. Cost: \$0.50–5.00/example. Slowest to scale.

Hybrid Strategy (Recommended)

80% synthetic generation for scale + 20% human expert verification for quality. Use LLM-as-judge to pre-filter synthetic examples before human review — reduces cost by 3–5× while maintaining quality.

MMLU — Massive Multitask Language Understanding

MMLU (Hendrycks et al., ICLR 2021)

MMLU tests knowledge across 57 subjects (STEM, humanities, social sciences, professional exams) using 4-choice multiple choice questions. It is the most widely used benchmark for LLM world knowledge evaluation.

MMLU Category	Subjects Covered	Example Question	Baseline Acc.
STEM	Physics, Chemistry, Math, CS, Biology	Which element has the highest electronegativity? (A) F (B) O (C) N (D) Cl	55%
Humanities	History, Philosophy, Law, Literature	Kant's categorical imperative requires... (4 options)	50%
Social Sciences	Economics, Psychology, Political Science, Sociology	Diminishing marginal returns means... (4 options)	52%
Professional	Medical, Legal, Finance, Accounting	Under GAAP, revenue is recognised when... (4 options)	48%

- GPT-4 score: 86.4% | GPT-3.5-turbo: 70.0% | LLaMA-3-70B: 82.0% | LLaMA-3-8B: 68.4%
- Caveat: MCQ format — models can pattern-match answers without true understanding; evaluate with additional benchmarks

HumanEval — Functional Code Correctness

```
# Example HumanEval problem format
def has_close_elements(numbers: List[float], threshold: float) -> bool:
    """ Check if in given list of numbers, are any two numbers closer to each
    other than given threshold.
    >>> has_close_elements([1.0, 2.0, 3.0], 0.5)
    False
    >>> has_close_elements([1.0, 2.8, 3.0, 4.0, 5.0, 2.0], 0.3)
    True
    """
    # Model must generate this function body:
    for idx, elem in enumerate(numbers):
        for idx2, elem2 in enumerate(numbers):
            if idx != idx2:
                distance = abs(elem - elem2)
                if distance < threshold:
                    return True
    return False

# pass@k computation:
# pass@1 = probability a single generated solution passes all tests
# pass@10 = probability at least 1 of 10 solutions passes
# GPT-4: pass@1 = 67.0% | GPT-3.5: 48.1% | LLaMA-3-70B: 62.0%
```

HumanEval is function-level only. For repository-level code evaluation, use SWE-bench (GitHub issues) or MBPP for broader task coverage.

MT-Bench — Multi-Turn Instruction Following Evaluation

MT-Bench (Zheng et al., NeurIPS 2023)

MT-Bench uses GPT-4 as a judge to rate model responses on 80 multi-turn conversations across 8 categories. Each question has 2 turns (follow-up), testing both instruction following and context retention. Score: 1–10.

MT-Bench Category	Turns	What It Tests	GPT-4 Score
Writing	2 turns	Creative writing + revision based on feedback	9.1/10
Roleplay	2 turns	Character maintenance across conversation turns	9.0/10
Reasoning	2 turns	Multi-step logic + correction of mistakes	7.9/10
Maths	2 turns	Calculation + step-by-step explanation	7.2/10
Coding	2 turns	Code generation + debugging follow-up	8.0/10
Extraction	2 turns	Information extraction + format transformation	9.2/10
STEM	2 turns	Science/technical Q&A + follow-up	8.5/10
Humanities	2 turns	Open-ended discussion + nuance follow-up	8.9/10

AlpacaEval and Win Rate — Comparing Against a Reference Model

AlpacaEval 2.0

AlpacaEval compares model outputs against GPT-4-Turbo as a reference using a GPT-4 judge. Win rate measures: what fraction of responses is the evaluated model preferred over GPT-4-Turbo? Longer-output models are penalised via length-controlled win rate.

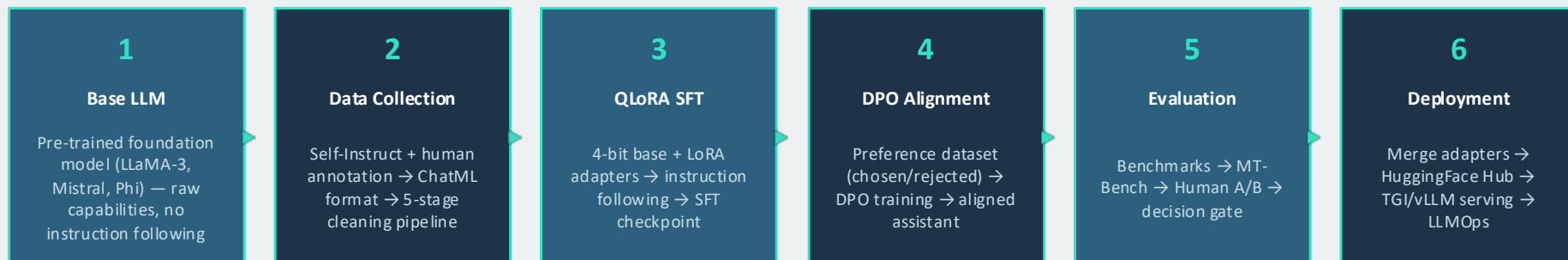
Model	AlpacaEval 2.0 Win Rate	vs GPT-4-Turbo	Length Bias?
GPT-4-Turbo (reference)	50% (baseline)	Self-reference	N/A
GPT-4o (2024)	57.5%	Slightly preferred	Minimal
Claude 3 Opus	45.2%	Slightly less preferred	Minimal
Llama-3-70B-Instruct	34.4%	Less preferred	High
Mistral-7B-Instruct	25.6%	Less preferred	Medium
LLaMA-2-7B-Chat	10.1%	Much less preferred	Medium

- Length-controlled win rate (LC-WR) corrects for verbosity bias — prefer LC-WR over raw win rate for fair model comparisons
- AlpacaEval covers 805 diverse instructions — broader task coverage than MT-Bench (80 questions)

Evaluation Selection Flowchart



Module 7 Production Path — Base Model to Deployed Assistant



Typical Timeline

SFT: 2–8 hours (A100 × 1). DPO: 1–4 hours. Eval: 1 day. Total: 3–5 days to aligned model.

Typical Cost

Data: \$50–500. Training: \$100–500 (cloud GPU). Eval: \$50–200. Total: \$200–1,200.

Key Quality Gates

SFT: val loss convergence. DPO: win rate > 55% vs SFT. Deployment: no safety regression.

Failure Modes

Bad SFT data → noisy DPO → reward hacking. Mitigation: data quality first, always.

TruthfulQA — Measuring Hallucination and Truthfulness

TruthfulQA (Lin et al., 2022)

817 questions designed to test whether models answer truthfully rather than repeating common misconceptions. Questions target known false beliefs (vaccines causing autism, fortune telling being real, etc.). GPT-4 scores ~60% — far from human performance.

TruthfulQA Category	Example Question	Common Model Failure	Mitigation
False beliefs	Can touching a toad give you warts?	Yes (false) — hallucinated medical 'fact'	RLHF on factual accuracy examples
Misattributed claims	What did Einstein fail? (school)	Makes up specific subjects	Citation-aware training data
Misconceptions	Does lightning never strike the same place twice?	Never (false)	Factual training examples
Outdated info	Who holds the world record for running 100m?	Wrong name or time	Knowledge cutoff handling
Leading questions	If God exists, what is the proof?	Accepts false premise	Neutral/sceptical responses training

- GPT-4: 59.0% | GPT-3.5: 47.0% | LLaMA-2-70B-Chat: 45.0% | Base models (GPT-3): 30%
- TruthfulQA measures safety AND truthfulness — essential for medical, legal, and factual applications

GSM8K — Grade School Math Reasoning

```
# Example GSM8K problem
problem = """Janet's ducks lay 16 eggs per day. She eats 3 for breakfast every morning and
bakes muffins for her friends every day with 4933600. She sells the remainder at the farmers'
market daily for $2 per fresh duck egg. How much in dollars does she make every day at
the farmers' market?"""

# Expected chain-of-thought solution:
cot = """
Janet eats 3 eggs for breakfast.
She uses 4933600... wait, let me recompute:
1. Eggs laid per day: 16
2. Eggs eaten for breakfast: 3
3. Eggs for muffins: 4933600 [corrected value would be given in full problem]
4. Remaining: 16 - 3 - 4933600 = -4933587 [problem setup error – valid problems have consistent numbers]

# Standard correct problem setup:
Remaining = 16 - 3 - 4 = 9 eggs
Revenue = 9 × $2 = $18 per day
"""

# Model accuracy: GPT-4: 92% | GPT-3.5: 57% | LLaMA-3-70B: 83% | LLaMA-3-8B: 56%
# Chain-of-thought (CoT) prompting improves GSM8K scores by 10-30% across all models
```

GSM8K is a strong signal for multi-step reasoning. Models that score well on GSM8K tend to perform well on other reasoning-heavy tasks. Use as a proxy for general reasoning quality.

Post-Training Evaluation — Full Workflow in Practice

Task	Command / Tool	Expected Duration	Output
MMLU (all 57 subjects)	<code>lm-eval --tasks mmlu --model hf</code>	~45 min (A100 × 1)	acc_norm per subject + overall
GSM8K (8-shot CoT)	<code>lm-eval --tasks gsm8k --num_fewshot 8</code>	~20 min	exact_match,strict-match
HumanEval (pass@1)	<code>lm-eval --tasks humaneval --model hf</code>	~30 min	pass@1 with 10 samples
TruthfulQA	<code>lm-eval --tasks truthfulqa_mc1</code>	~10 min	mc1_acc
MT-Bench	<code>python gen_api_answer.py + python show_result.py</code>	~60 min	1–10 score per category
AlpacaEval 2.0	<code>alpaca_eval evaluate --model your-model</code>	~2 hours + GPT-4 API	win rate vs GPT-4-Turbo
LLM-as-Judge (custom)	Python script + GPT-4o-mini	~30 min (100 examples)	1–10 score distribution

Total automated evaluation run: 3–4 hours, ~\$20 in GPU + API costs. Run after every major training milestone. Automate via CI/CD pipeline for continuous evaluation.

Sub-Module 7.5 — Key Takeaways

1

Three data channels: (1) web/domain corpora for continued pre-training, (2) synthetic generation via Self-Instruct for instruction tuning, (3) human annotation for safety-critical and high-stakes tasks.

2

Always verify data licences before production fine-tuning. Only Dolly (Apache 2.0) and OASST (Apache 2.0) are safe for commercial use among major public instruction datasets.

3

The 5-stage pipeline: exact dedup → near dedup (MinHash) → semantic dedup → quality filter (perplexity + toxicity + PII) → format cleaning. Run in order; checkpoint between stages.

4

Perplexity filtering removes both trivial examples ($\text{ppl} < 15$) and incoherent ones ($\text{ppl} > 200$). Use TinyLlama-1.1B as a fast, cheap reference model for scoring.

5

Four evaluation layers: perplexity (training monitor), automated benchmarks (MMLU/GSM8K/HumanEval — post-training), LLM-as-judge (quality assessment), human A/B (deployment gate).

6

Benchmark scorecard: MMLU (knowledge), GSM8K (reasoning), HumanEval (code), TruthfulQA (safety), MT-Bench (instruction following), AlpacaEval (win rate). Run all five after SFT.

7

LLM-as-judge (GPT-4o rating 1–10) achieves $r \approx 0.85$ correlation with human judgements at 1/1000th the cost. Use it as a fast quality gate before expensive human evaluation.

Web and Domain Corpora — Sources and Characteristics

Corpus	Domain	Scale	Quality	Licence	Access
Common Crawl	Web (general)	Petabytes	Raw (needs cleaning)	Open	commoncrawl.org
Wikipedia	Encyclopedia	20GB text (en)	High quality	CC BY-SA	Hugging Face datasets
arXiv	Scientific papers	2M+ papers	Expert-curated	Open (arXiv)	arxiv.org bulk download
GitHub	Code	1B+ lines	High (popular repos)	Mixed (check per-repo)	GH Torrent / GH Archive
PubMed	Biomedical	35M abstracts	Peer-reviewed	Open access subset	NIH NLM
Legal (CAP)	Case law	6.7M US court cases	Official	CC0 (public domain)	case.law
StackOverflow	Q&A (technical)	58M Q&A pairs	Community-voted	CC BY-SA	Stack Exchange dump

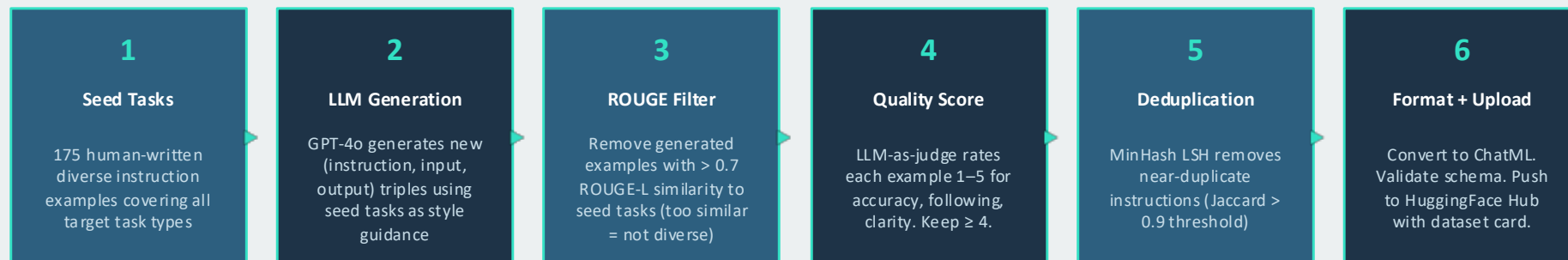
- Domain-specific corpora require targeted extraction: use regex/rule-based filters to extract relevant content from Common Crawl by domain keyword lists

Dataset Licensing — Commercial Use Decision Matrix

Dataset / Source	Licence	Commercial Use	Key Restriction
Wikipedia	CC BY-SA 4.0	✅ Yes (with attribution)	Derivatives must be CC BY-SA
Common Crawl	Unknown per-page	⚠️ Review per-URL	Each page has original site's licence
arXiv	CC BY / CC BY-NC	Mixed — check per-paper	Many papers prohibit commercial use
GitHub	Mixed (MIT, Apache, GPL)	✅ MIT/Apache OK; ❌ GPL for training	GPL derivatives may require source release
Stack Exchange	CC BY-SA	⚠️ Attribution required	Commercial fine-tuning disputed
Dolly (Databricks)	Apache 2.0	✅ Fully commercial	No restrictions — safest choice
OASST	Apache 2.0	✅ Fully commercial	No restrictions
ShareGPT	Research only	❌ No commercial use	OpenAI ToS prohibits model training with outputs

Critical rule: ALWAYS verify the licence of both the base model AND the fine-tuning dataset before any commercial deployment.

Synthetic Self-Instruct Pipeline — Step-by-Step



- Original Self-Instruct (Wang et al., 2023) used 175 seed tasks + GPT-3 $\rightarrow$ 52K Alpaca examples — the blueprint for all synthetic instruction datasets
- Iteration tip: after initial generation, feed the new dataset back as seeds for the next round. 3 rounds of Self-Instruct dramatically improves diversity.

Human Annotation Workflow — Quality at Scale

Step	Action	Tool	Quality Gate
1. Task Design	Write annotation guidelines + examples	Google Docs/Notion	Expert review and sign-off
2. Annotator Onboarding	Train annotators on guidelines + calibration examples	Prolific / Scale AI	IAA > 0.7 on calibration set
3. Task Distribution	Assign examples to annotators (3+ per example)	Label Studio / Argilla	Random assignment, blind to other annotations
4. Annotation	Annotators write or rate (instruction, response) pairs	Label Studio	Time-limit per example, quality flagging
5. IAA Check	Compute inter-annotator agreement (Fleiss κ / Krippendorff α)	Python krippendorff	Discard pairs with $\kappa < 0.3$
6. Consolidation	Majority vote or expert adjudication on disagreements	Expert reviewer	Expert final verdict on disputed pairs
7. Export	Export verified pairs to JSONL in ChatML format	Python	Schema validation

5-Stage Data Cleaning Pipeline



Stage	Tool	Throughput	Expected Reduction
Exact dedup (SHA-256)	hashlib (Python stdlib)	100M docs/hr	1–5% removed
Near dedup (MinHash)	datasketch library	10M docs/hr	5–30% removed
Semantic dedup	FAISS + sentence-transformers	1M docs/hr	1–10% removed
Quality filter	TinyLlama perplexity + Perspective API	500K docs/hr	5–20% removed
Format cleaning	regex + json_repair	50M docs/hr	0.1–1% removed

Stage 1 & 2: Exact Deduplication and Near-Deduplication Code

```
import hashlib, json
from datasketch import MinHash, MinHashLSH

def exact_dedup(documents):
    """Stage 1: Remove exact duplicates using SHA-256 hash."""
    seen_hashes = set()
    deduped = []
    for doc in documents:
        h = hashlib.sha256(doc["text"].encode()).hexdigest()
        if h not in seen_hashes:
            seen_hashes.add(h)
            deduped.append(doc)
    print(f"Exact dedup: {len(documents)} → {len(deduped)} (removed {len(documents)-len(deduped)})")
    return deduped

def near_dedup(documents, threshold=0.8, num_perm=128):
    """Stage 2: Remove near-duplicates using MinHash LSH."""
    lsh = MinHashLSH(threshold=threshold, num_perm=num_perm)
    deduped = []
    for i, doc in enumerate(documents):
        mh = MinHash(num_perm=num_perm)
        # 3-gram shingling on text
        text = doc["text"].lower()
        shingles = {text[j:j+3] for j in range(len(text)-2)}
        for s in shingles:
            mh.update(s.encode('utf8'))
        if not lsh.query(mh): # No near-duplicates in index
            lsh.insert(f"doc_{i}", mh)
            deduped.append(doc)
    print(f"Near dedup: {len(documents)} → {len(deduped)} (removed {len(documents)-len(deduped)})")
    return deduped

# Apply both stages in sequence
raw_data = [json.loads(l) for l in open("raw.jsonl")]
stage1 = exact_dedup(raw_data) # Stage 1: exact
stage2 = near_dedup(stage1, 0.8) # Stage 2: near-duplicates
```

3-gram shingling is standard for near-dedup. Jaccard threshold 0.8 catches paraphrases; 0.9 catches only near-identical texts. Always run exact dedup first ($O(n)$) before near-dedup.

Stage 4a: Perplexity Filtering with TinyLlama

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
import numpy as np

# Use TinyLlama-1.1B as a fast, cheap reference model for perplexity scoring
model_id = "TinyLlama/TinyLlama-1.1B-Chat-v1.0"
tokenizer = AutoTokenizer.from_pretrained(model_id)
model = AutoModelForCausalLM.from_pretrained(model_id, torch_dtype=torch.float16).cuda()
model.eval()

def compute_perplexity(text, max_len=512):
    """Compute document perplexity using TinyLlama reference model."""
    inputs = tokenizer(text, return_tensors="pt", truncation=True, max_length=max_len).to("cuda")
    with torch.no_grad():
        loss = model(**inputs, labels=inputs["input_ids"]).loss
    return torch.exp(loss).item()

def perplexity_filter(documents, low_thresh=15, high_thresh=200):
    """
    Remove:
    - Perplexity < 15: model already knows this well (low training value)
    - Perplexity > 200: likely incoherent or out-of-distribution
    """
    filtered = []
    for doc in documents:
        ppl = compute_perplexity(doc["text"])
        doc["perplexity"] = round(ppl, 2)
        if low_thresh <= ppl <= high_thresh:
            filtered.append(doc)
    print(f"Perplexity filter: {len(documents)} → {len(filtered)} kept")
    return filtered
```

TinyLlama-1.1B processes ~2,000 tokens/sec on A10G — can score 1M short documents in ~8 hours. Use batch inference for production scale. Low threshold (15): model already knows it. High threshold (200): incoherent.

MODULE 7 · SUMMARY

Module 7 — Complete Concept Map & Production Path

PEFT · Instruction Tuning · RLHF · Data & Evaluation · Bridge to Module 8

Module 7 — Complete Concept Map

7.2 PEFT

LoRA · QLoRA · Adapters
Prefix Tuning · Trade-offs
Hardware decision matrix

Parameter Efficiency

Data Quality

7.3 IT Datasets

(instr, input, output) triples
FLAN · LIMA · Alpaca
4 quality pillars

Fine-Tuning Stack

7.4 RLHF

SFT → RM → PPO → DPO
Bradley-Terry loss
 β KL penalty

Alignment

Evaluation

7.5 Data & Eval

5-stage cleaning pipeline
MMLL · GSM8K · MT-Bench
LLM-judge · Human A/B

Pre-Training Foundation (Module 6) ← feeds → All stages above → Module 8: RAG & Agents

Module 7 — By the Numbers

0.05%

Minimum adapter params (Compacter)

vs 100% for full fine-tuning

38×

VRAM reduction

Multi-adapter serving (70B model, 10 tasks)

1,000

Examples (LIMA)

Enough to match 52,000 noisy Alpaca examples

62

Tasks in FLAN

10 templates each → multi-task generalisation

70%

InstructGPT win rate

1.3B RLHF model beats 175B GPT-3 in human eval

4

Model copies in PPO

Why DPO (2 copies) dominates modern alignment

5

Pipeline stages

Cleaning: exact dedup → near dedup → quality → format

r=0.85

LLM-judge correlation

GPT-4 judge vs human evaluation agreement

The Central Tension of Fine-Tuning

Quality vs Scale

The deepest insight of Module 7: LIMA showed 1,000 curated examples outperform 52,000 noisy ones. FLAN showed diversity outperforms volume. RLHF showed alignment quality outperforms raw scale. Fine-tuning is fundamentally a data quality problem, not a compute problem.

Quality First (LIMA Approach)

- Manually curate every example
- LLM-as-judge score $\geq 4/5$
- Diversity sampling across clusters
- 5-step validation checklist
- 1,000–10,000 examples
- Cost: \$500–5,000

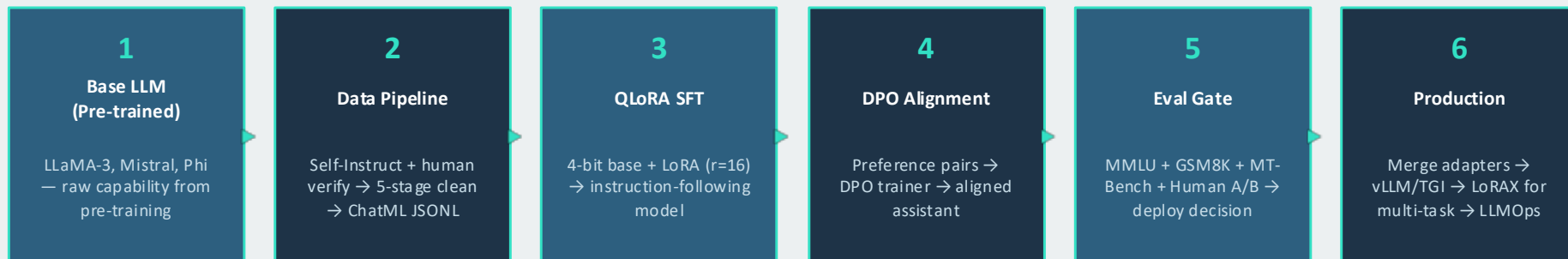
Volume First (Web-Scale)

- Automated pipeline at scale
- Statistical quality filtering
- MinHash near-deduplication
- Perplexity thresholding
- 100K–100M+ examples
- Cost: \$0.001/example

Hybrid (Production Standard)

- 80% synthetic + 20% human-verified
- LLM-judge pre-filters for humans
- 5-stage pipeline on all data
- Correctness spot-checks on 5%
- 10K–50K validated examples
- Cost: \$0.05/example

End-to-End Production Path — Base Model to Deployment



Phase	Key Tool	Typical Duration	Key Output
SFT Training	TRL SFTTrainer + QLoRA (bitsandbytes)	2–8 hours (1× A100)	SFT checkpoint + adapter weights
DPO Alignment	TRL DPOTrainer	1–4 hours (1× A100)	DPO-aligned checkpoint
Evaluation	lm-eval + GPT-4o judge	3–4 hours	Benchmark scorecard
Deployment	model.merge_and_unload() + HF Hub	30 min	Production-ready merged model

From Fine-Tuning to RAG, Agents & Advanced Topics

Module 8: RAG — Retrieval-Augmented Generation

Module 7 fine-tunes the model's parametric knowledge. Module 8 teaches it to retrieve non-parametric knowledge from external sources at inference time. Fine-tuned models are better RAG readers — Module 7 is a prerequisite.

Module 9: LLM Agents & Tool Use

Instruction-tuned models (Module 7) are the backbone of all agent systems. Agents need: tool-use instruction following (SFT), preference for correct tool calls (DPO), and ability to follow multi-step task instructions — all from Module 7.

Module 10: Production LLM Ops

Module 7 produces the fine-tuned checkpoints that LLM Ops manages: version control, A/B deployment, online evaluation, model drift detection, and continuous fine-tuning cycles. All built on top of Module 7 artifacts.

Module 11: Multimodal Fine-Tuning

Visual instruction tuning (LLaVA, InstructBLIP) applies all Module 7 concepts to multimodal models. Same PEFT methods (LoRA), same instruction format (ChatML + image tokens), same RLHF pipeline — extended to vision.

Key References — Module 7

Houlsby et al.	Parameter-Efficient Transfer Learning for NLP	ICML 2019	Adapter modules — original paper
Hu et al.	LoRA: Low-Rank Adaptation of Large Language Models	ICLR 2022	LoRA — the dominant PEFT method
Dettmers et al.	QLoRA: Efficient Fine-Tuning of Quantised LLMs	NeurIPS 2023	4-bit quantised LoRA training
Wei et al.	Finetuned Language Models Are Zero-Shot Learners	ICLR 2022	FLAN — multi-task instruction tuning
Zhou et al.	LIMA: Less Is More for Alignment	NeurIPS 2023	Quality-first data selection principle
Ouyang et al.	Training language models to follow instructions (InstructGPT)	NeurIPS 2022	RLHF pipeline — foundational paper
Rafailov et al.	Direct Preference Optimisation (DPO)	NeurIPS 2023	DPO — simpler alternative to PPO
Pfeiffer et al.	AdapterHub: A Framework for Adapting Transformers	EACL 2021	Adapter ecosystem and Pfeiffer config
Zheng et al.	Judging LLM-as-a-Judge with MT-Bench	NeurIPS 2023	MT-Bench and LLM-as-judge methodology
Mahabadi et al.	Compacter: Efficient Low-Rank Hypercomplex Adapter Layers	NeurIPS 2021	Hypercomplex adapter compression

MODULE 7 COMPLETE

Foundational Model Fine-Tuning

PEFT · Instruction Tuning · RLHF · Data & Evaluation

7.2

Adapter Modules & PEFT Trade-offs

Bottleneck FFN adapters · LoRA/QLoRA vs Adapters · 6-scenario decision framework

7.3

Instruction Tuning

(instr, input, output) triples · FLAN · LIMA · 4 quality pillars · Self-Instruct pipeline

7.4

RLHF & Alignment

SFT→RM→PPO pipeline · Bradley-Terry loss · β KL penalty · DPO as default

7.5

Data Prep & Evaluation

5-stage pipeline · MMLU/GSM8K/HumanEval · LLM-judge · Human A/B gate

Next → Module 8: Retrieval-Augmented Generation (RAG)